WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Documentation
WAC (Windows Artefacts Collector)

Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

WAC — Windows Artefacts Collector

Documentation utilisateur

Version du logiciel	2.0.0
Édition du document	septembre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC)
	École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document décrit l'emploi de WAC : préparation, exécution d'une collecte, lecture et vérification des résultats, et traces laissées sur la machine examinée. Les mesures citées ont été relevées sur une machine virtuelle Windows 11 de test ; elles donnent des ordres de grandeur, pas des engagements.

Table des matières

1 Présentation **4**

1.1 À quoi sert WAC 4

1.2 Le principe directeur : une empreinte minimale 4

1.3 Ce que WAC ne fait pas 4

2 Principes de fonctionnement **6**

2.1 Lecture directe du disque 6

2.2 Consigne et travail 6

2.3 Ruches du registre et journaux de transaction 7

2.4 L'extraction en deux temps 7

2.5 Ce qui reste lu « à chaud » 7

3 Préparer une collecte **9**

3.1 Prérequis 9

3.2 Le support de collecte 9

3.3 Avant d'intervenir 10

4 Lancer une collecte **11**

4.1 Ligne de commande 11

4.2 Fichier de configuration : `wac.yml` 11

4.3 Options 14

4.4 Exemples 16

4.5 L'option `--binary` 16

4.6 Déroulement à l'écran 18

4.7 Durée 19

5 Collecter ici, convertir ailleurs **20**

5.1 Pourquoi séparer 20

5.2 Les réglages de la collecte 21

5.3 Les deux commandes 21

5.4 Ce que prend `--collect` 21

5.5 Ce que fait `--convert` . . . 22
5.6 Limites . . . 23

6 Les résultats **24**
6.1 Arborescence . . . 24
6.2 Le manifeste de consigne . . . 24
6.2.1 En-tête . . . 24
6.2.2 Chaque pièce . . . 24
6.3 Vérifier l'intégrité d'une collecte . . . 26
6.3.1 Le sceau . . . 26
6.3.2 Chaque pièce . . . 26
6.4 Le journal d'investigation . . . 29
6.5 Les fichiers d'artefacts . . . 30
6.5.1 Système et comptes . . . 30
6.5.2 Exécution de programmes . . . 31
6.5.3 Fichiers et navigation . . . 39
6.5.4 Périphériques . . . 41
6.5.5 Journaux d'événements (`--events`) . . . 41

7 Lire les données **43**
7.1 Horodatages . . . 43
7.2 Noms des comptes . . . 43
7.3 Noms des GUID . . . 44
7.4 Un champ absent n'est pas un champ vide . . . 44
7.5 Artefact non collecté . . . 44
7.6 Données non décodées . . . 44
7.7 Chemins . . . 45

8 Traces laissées sur la machine examinée **46**
8.1 Ce que WAC fait encore . . . 46
8.2 Ce que WAC ne fait plus . . . 47
8.3 Ce qu'aucun outil ne peut éviter . . . 47

9 Limites connues **48**

10 Dépannage **49**

A Glossaire **50**

CHAPITRE 1

Présentation

1.1 À quoi sert WAC

WAC prélève, sur un poste Windows en fonctionnement, les artefacts utiles à une investigation numérique : base de registre, journaux d'événements, fichiers Prefetch, raccourcis et listes de sauts, tâches planifiées, services, comptes, processus et sessions. Il les restitue sous forme de fichiers **JSON**, prêts à être analysés ou versés dans une chaîne de traitement.

Il s'exécute depuis une clé USB, sans installation, et écrit tout ce qu'il produit sur cette clé.

1.2 Le principe directeur : une empreinte minimale

Examiner une machine la modifie. Chaque outil lancé laisse des traces : fichiers ouverts dont la date de dernier accès change, services sollicités qui écrivent dans les journaux, clés de registre lues par l'API. Ces traces se mêlent à celles que l'on cherche.

WAC est conçu pour que ce qu'il laisse derrière lui soit aussi proche de rien que la tâche le permet, et pour que **ce qu'il ne peut pas éviter soit écrit**. Concrètement :

- le disque est lu **directement**, en lecture seule, sans ouvrir aucun fichier par le système : aucune date d'accès n'est modifiée ;
- aucune clé du registre de la machine n'est ouverte : les ruches sont copiées, puis lues sur la copie ;
- aucun cliché instantané (VSS), aucun appel COM ou WMI, aucun service de journalisation n'est sollicité ;
- rien n'est écrit sur le disque examiné ;
- chaque opération est consignée, avec la trace qu'elle est censée laisser, dans `investigation.json`.

1.3 Ce que WAC ne fait pas

- WAC **n'analyse pas** : il prélève et met en forme. L'interprétation revient à l'analyste.
- Il ne réalise pas d'image complète du disque : il extrait les fichiers porteurs d'artefacts.
- Il ne capture pas la mémoire vive. Les seules données volatiles relevées sont la liste des

processus, les sessions ouvertes et l’état des services.

CHAPITRE 2

Principes de fonctionnement

2.1 Lecture directe du disque

WAC ouvre le volume (`\\.\C:`) en lecture seule, parcourt lui-même la table des fichiers NTFS (`$MFT`) et en extrait le contenu des fichiers utiles. Le système ne voit aucune ouverture de fichier : ni date d'accès mise à jour, ni parcours de répertoire, ni verrou.

Cette lecture prend en charge les particularités de NTFS que Windows masque d'ordinaire :

Compression NTFS
: Windows 11 compresse notamment le dossier des journaux d'événements. WAC les décompresse lui-même.

Compact OS (WOF)
: Les binaires système de Windows 10 et 11 sont stockés compressés dans un flux caché ; vus par l'API, ils paraissent ordinaires. WAC les décompresse (variantes XPRESS).

Attributs éclatés
: Un fichier très fragmenté voit ses informations réparties sur plusieurs enregistrements ; WAC les réunit.

Fichiers préalloués
: Au-delà de la longueur de données valides, NTFS rend des zéros, même si le disque contient encore les restes d'anciens fichiers. WAC fait de même : le contenu extrait est celui que verrait toute acquisition ordinaire, et son empreinte aussi.

2.2 Consigne et travail

Une pièce à conviction numérique ne s'examine jamais sur elle-même : on en fait une copie, on la scelle, et l'on travaille sur une seconde copie. Si l'analyse abîme quelque chose, la copie scellée demeure et l'opération peut être refaite.

WAC applique cette procédure à **tous** les fichiers extraits, y compris ceux qu'il ne modifie pas :

`exhibits/`
: les copies brutes, telles que lues sur le volume. Une fois recopié dans la copie de travail, ce répertoire n'est plus touché, ni en écriture ni en lecture : seule la vérification de son intégrité, avant la recopie d'une conversion, le relit. Il contient le manifeste qui identifie chaque pièce, et le sceau du manifeste.

`working/`
: les copies de travail, recopiées depuis la consigne et vérifiées par empreinte — toutes, `$MFT`, journal des modifications et binaires prélevés compris, ce qui double leur place sur le support. C'est ici que sont faites les seules modifications (rejeu des journaux de transaction des ruches), et c'est ici que lisent tous les collecteurs.

> **À retenir**
> Chaque fichier extrait est donc écrit **deux fois** sur le support de collecte. Prévoyez la place correspondante (voir section 3.2).

2.3 Ruches du registre et journaux de transaction

Une ruche copiée sur une machine en fonctionnement ne contient pas les dernières modifications : elles sont encore dans ses journaux de transaction (`.LOG1`, `.LOG2`). WAC extrait ces journaux et les **rejoue sur la copie de travail**, pour que l'analyse porte sur l'état réel de la machine.

Tout ce que WAC écrit sur une copie est réversible : avant le rejeu, le contenu d'origine de chaque page remplacée est sauvegardé dans un journal d'annulation (`<ruche>.undo`). La copie brute, elle, reste intacte dans la consigne.

2.4 L'extraction en deux temps

L'emplacement des ruches de chaque utilisateur est inscrit dans la ruche `SOFTWARE`. WAC extrait donc d'abord les ruches de la machine (`SYSTEM`, `SOFTWARE`, `SAM`, `Amcache`), lit la liste des profils dans la copie de `SOFTWARE`, puis extrait les ruches de chaque profil (`ntuser.dat`, `usrClass.dat`).

Rien n'est supposé sur `C:` : le lecteur système est détecté, et un profil situé sur un autre disque (par exemple `D:\Users\...`) est lu sur son propre volume.

2.5 Ce qui reste lu « à chaud »

Quelques informations décrivent l'instant même de la collecte et ne figurent dans aucun fichier. Elles sont lues en direct, en une seule requête chacune, et enregistrées telles que lues dans la consigne (`exhibits\live`) et en même temps dans la copie de travail, où la conversion les relit :

- la liste des **processus** en cours (instantané du noyau) ;
- les **sessions** ouvertes ;
- l'**état courant des services** (la configuration, elle, est lue dans la ruche copiée) ;

— l'**horloge** et l'heure de démarrage ;

— les **réglages** de la machine en marche : lecteur système, page de code, fuseau horaire, volumes et leurs numéros de série.

CHAPITRE 3

Préparer une collecte

3.1 Prérequis

— Windows 10 ou ultérieur, 64 bits. WAC le vérifie au lancement.

— Des **droits d'administrateur** : la lecture directe d'un volume les exige. WAC le vérifie également.

— Des volumes **NTFS**, **FAT** ou **exFAT**. Un fichier situé sur un autre système de fichiers (CD-ROM) ne peut pas être lu directement ; il est signalé comme non lu, jamais ouvert autrement.

3.2 Le support de collecte

WAC s'exécute depuis le support de collecte (en général une clé USB) et y écrit tous ses résultats. Choisissez un support **vide**, de préférence en USB 3 : l'écriture est le poste le plus coûteux d'une collecte.

La place nécessaire dépend du mode et des options. Deux emplacements se remplissent : la **consigne** (copies brutes scellées) et la **copie de travail**, qui ne reprend que ce que les collecteurs modifient ou lisent pendant la collecte (ruches, Prefetch, raccourcis, listes de sauts, binaires cités). Les binaires prélevés par `--collect --binary` et les métadonnées NTFS ne sont jamais recopiés : la conversion les lit dans la consigne même. Mesures sur la machine virtuelle de test (Windows 11 sans logiciel ajouté, 1 volume) :

Collecte	Consigne	Copie de travail	Durée
`--collect --events --binary` (tous les exécutables)	2,7 Go	0,3 Go	8 min 30

Les postes les plus lourds de la consigne :

Contenu	Consigne
Métadonnées NTFS : `$MFT` (380 Mo), `$LogFile` (64 Mo), `$UsnJrnl:$J` (35 Mo, sur 3,6 Go logiques : NTFS a libéré le début, qui n'est pas copié)	480 Mo
Binaires copiés : 2 853 non authentifiés sur 44 446 lus	1,2 Go
Journaux d'événements (`--events`)	170 Mo
Ruches, Prefetch, raccourcis, listes de sauts	180 à 230 Mo

Une machine utilisée depuis longtemps, ou chargée de logiciels, dépasse ces chiffres : la `$MFT` croît avec le nombre de fichiers, les binaires tiers non authentifiés avec les logiciels installés. Prévoir **au moins 5 Go libres** pour une collecte seule avec `--binary`.

WAC vérifie l'emplacement **avant la toute première écriture** et refuse de démarrer :

— si le répertoire de travail contient déjà des fichiers — une collecte antérieure serait analysée à la place de celle-ci ;

— si la place libre est manifestement insuffisante.

Si la place vient à manquer en cours de prélèvement des binaires, ceux-ci sont hachés sans être copiés, et le journal d'investigation l'indique.

3.3 Avant d'intervenir

— Utilisez un **dossier de sortie neuf** pour chaque machine (option `--output`).

— Notez l'heure de l'intervention et l'identité de l'opérateur : WAC les consigne aussi, mais une trace indépendante est utile.

— Gardez à l'esprit que brancher la clé et lancer WAC laissent des traces inévitables (chapitre 8) : elles sont connues et documentées, afin de ne pas être confondues avec l'activité étudiée.

CHAPITRE 4

Lancer une collecte

4.1 Ligne de commande

Ouvrez une invite de commandes **en tant qu'administrateur**, placez-vous sur le support de collecte, puis lancez :

```
WAC.exe [--collect | --convert=dossier] [--events] [--binary | --binary-all]
        [--threads=N] [--dump]
        [--output=dossier] [--loglevel=N] [--debug]
WAC.exe --update-trust[=dossier]
```

Toutes les options sont facultatives et désactivées par défaut.

4.2 Fichier de configuration : `wac.yml`

Version. WAC numérote ses versions `<majeur>.<mineure>.<correctif>` : majeur pour une refonte, mineure pour un ajout de fonctionnalités, correctif pour des corrections. La version se lit dans les propriétés de `WAC.exe` (onglet Détails), s'affiche au lancement, et est consignée dans `investigation.json` (`Tool.Version`) : une collecte dit par quelle version elle a été faite. Ce document décrit la version 2.0.0.

La dernière version compilée se trouve dans `bin` du dépôt : `WAC.exe`, le `wac.yml` de référence, les scripts qui vérifient une collecte (section 6.3) et `SHA256SUMS` pour vérifier le tout. Copier `WAC.exe` et `wac.yml` sur la clé de collecte suffit ; les scripts restent avec l'enquêteur.

Plutôt que de retaper les options à chaque collecte, la procédure s'écrit une fois dans `wac.yml`, à côté de `WAC.exe` : WAC l'applique d'abord, et une option donnée en ligne de commande la surcharge. `--write-config` écrit le fichier de référence, commenté — la procédure recommandée : binaires non vérifiés prélevés, journaux d'événements compris.

```
wac_version: 2.0.0      # version de WAC pour laquelle le fichier est écrit (--write-
    config)
binary: unverified      # none | unverified | all
threads: 0              # fils d'analyse des binaires, 0 : automatique
output: output
log_level: 0
dump: false
language: fr            # fr | en : langue de la console ; les fichiers restent en anglais
artefacts:              # une clé par artefact ; une clé absente vaut true
  activities_cache:        true   # Chronologie Windows : applications, fichiers, pages,
    presse-papiers (ActivitiesCache.db)
  amcache:                 true   # Amcache.hve : applications, exécutables et pilotes vus
  bam:                     true   # dernière exécution de chaque programme par utilisateur
```

```
    (BAM)
  bits:                    true   # qmgr.db : travaux BITS, URL, destinations, commandes
    de notification
  defender:                true   # Defender : journaux (MPLog, traces WPP brutes),
    historique des détections, quarantaine (copiée chiffrée)
  defender_configuration:  true   # exclusions et réglages de protection de Microsoft
    Defender
  devices:                 true   # périphériques USB et SCSI et leurs dates, portables,
    noms de volumes
  dpapi_master_keys:       true   # clés maîtresses DPAPI copiées chiffrées ; JSON :
    propriétaire, GUID, dates, jamais la clé
  emails:                  true   # Outlook (PST, OST) et Thunderbird (mbox) : messages,
    destinataires, dates ; corps et pièces jointes exportés
  error_reports:           true   # rapports d'erreurs (WER) : programmes plantés, avec
    leur chemin
  event_log_configuration: true   # état de chaque journal et session ETW de démarrage :
    actif, taille, rétention, accès (hors ligne)
  events:                  true   # journaux d'événements, avec leurs messages
  explorer_recent_docs:    true   # fichiers ouverts par chaque utilisateur (RecentDocs de
     l'Explorateur)
  fat_volumes:             true   # volumes FAT et exFAT, clés USB du suspect comprises :
    entrées, supprimées aussi
  firewall:                true   # règles et profils du pare-feu
  hosts:                   true   # le fichier hosts : noms détournés vers une adresse
  installed_programs:      true   # programmes installés (Uninstall) de la machine et de
    chaque utilisateur
  internet_settings:       true   # proxy des utilisateurs
  jump_lists:              true   # listes de sauts : fichiers et lieux ouverts, par
    application
  known_networks:          true   # réseaux auxquels la machine s'est connectée (
    NetworkList)
  local_groups:            true   # membres de chaque groupe local (SAM)
  log_file:                true   # $LogFile et $TxfLog : journaux de transactions NTFS et
     TxF, copie brute seule
  mft:                     true   # $MFT : chaque fichier, présent ou supprimé, et ses
    dates
  mounted_devices:         true   # volumes et lettres de lecteur connus de la machine (
    MountedDevices)
  mount_points:            true   # volumes et partages vus par chaque utilisateur (
    MountPoints2)
  mru:                     true   # fichiers ouverts ou enregistrés par les boîtes de
    dialogue communes
  mru_apps:                true   # programmes qui ont ouvert ou enregistré ces fichiers
  muicache:                true   # noms des programmes lancés par chaque utilisateur
  network:                 true   # en direct : connexions, voisins, interfaces, routes
  network_configuration:   true   # cartes réseau, dernière configuration IPv4/IPv6, baux
    DHCP (hors ligne)
  network_shares:          true   # partages réseau de la machine, chemin et droits
  ntfs_volumes:            true   # $Boot et $UsnJrnl:$Max : numéro de série et géométrie,
     création du journal
  office_trust_records:    true   # documents Office approuvés, macros activées
  operating_system:        true   # version, installation, nom de la machine, durée de
    fonctionnement
```

```
  persistence:             true   # persistance du registre : IFEO, AppInit, Winlogon, LSA
    , …COM
  powershell_history:      true   # PSReadLine : les commandes tapées dans PowerShell
  prefetch:                true   # Prefetch : programmes lancés, compteur, huit dernières
     exécutions, fichiers chargés
  processes:               true   # en direct
  program_usage:           true   # programmes lancés ou utilisés (FeatureUsage,
    JumplistData, compatibilité, épinglés)
  rdp_client:              true   # Bureau à distance sortant : serveurs, fichiers .rdp,
    tuiles du cache bitmap
  rdp_server:              true   # Bureau à distance entrant : authentifications,
    sessions, adresses source
  recent_documents:        true   # Recent : raccourcis vers les fichiers ouverts par
    chaque utilisateur
  recycle_bin:             true   # $I des corbeilles : quoi, par qui, quand
  registry_backups:        true   # RegBack : anciennes ruches de la machine, copie brute
    seule
  root_certificates:       true   # certificats racines de la machine et des utilisateurs
  run_keys:                true   # clés Run et RunOnce de la machine et de chaque
    utilisateur
  scheduled_tasks:         true   # tâches planifiées : commandes, déclencheurs, auteurs,
    dates
  search_index:            true   # index de recherche Windows : chemin, dates, taille, ré
    sumé de chaque élément indexé
  security:                true   # ruche SECURITY : noms des secrets LSA, connexions en
    cache, audit
  security_descriptors:    true   # $Secure:$SDS : propriétaire et droits de chaque
    fichier, chaque descripteur une fois
  services:                true   # services et pilotes
  sessions:                true   # en direct
  setupapi:                true   # setupapi.dev.log : installations de périphériques
  shellbags:               true   # dossiers parcourus par chaque utilisateur dans l'
    Explorateur
  shimcache:               true   # cache de compatibilité des applications (
    AppCompatCache)
  shortcuts:               true   # raccourcis hors de Recent : menu Démarrer, Bureaux,
    Quick Launch, barre des tâches
  srum:                    true   # SRUDB.dat : réseau et ressources par application et
    utilisateur, 30 à 60 jours
  startup_folders:         true   # dossiers de démarrage : ce qui s'exécute à l'ouverture
     de session
  thumbcache:              true   # caches des miniatures et des icônes : ce que chacun a
    vu, même supprimé ; images exportées
  typed_paths:             true   # chemins tapés dans l'Explorateur
  usbstor:                 true   # périphériques de stockage USB (Enum\USBSTOR)
  userassist:              true   # programmes lancés depuis l'Explorateur, avec compteur
    et date
  users:                   true   # comptes locaux (SAM) : connexions, échecs, indicateurs
    , condensats
  user_shell_folders:      true   # où sont vraiment le Bureau, les Documents, les Télé
    chargements de chacun
  usn_journal:             true   # $UsnJrnl:$J : créations, renommages, suppressions
  web_client:              true   # navigateurs Chromium et Firefox : visites, télé
```

```
    chargements, recherches ; leurs bases copiées brutes
  wmi:                    true   # dépôt WMI lu hors ligne : abonnements (filtres,
    consommateurs, liaisons)
  word_wheel_query:       true   # recherches de l'Explorateur
  zone_identifiers:       true   # flux Zone.Identifier : d'où viennent les fichiers télé
    chargés
```

— **Strict** : une clé inconnue, une valeur hors de celles admises (`true`/`false` seulement, pas `yes`), une clé en double, une liste : la collecte est refusée avant de commencer, avec la ligne fautive. Une clé absente garde sa valeur par défaut : `true`.

— **Versionné** : `wac_version` porte la version de WAC pour laquelle le fichier a été écrit (`--write-config` l'y met). Un fichier d'une autre version, ou sans version, est refusé avant toute lecture : il pourrait manquer des réglages ajoutés depuis, ou en porter qui n'existent plus. Le refaire avec `--write-config`.

— **Une clé par artefact**, aucun prélevé d'office. Les ruches, les journaux d'événements et la `$MFT` ne sont pas des artefacts mais des *dépendances* : prises dès qu'un artefact demandé les lit, et seulement alors. Les journaux servent à `events` et à `rdp_server` ; les ruches des utilisateurs à `muicache`, `userassist`, `run_keys`, `shellbags`, `mru`, `mru_apps` et aux autres artefacts lus dans `NTUSER.DAT` ; `Amcache.hve` au seul `amcache`. SYSTEM, SOFTWARE, SAM, SECURITY et DEFAULT sont toujours prises : elles donnent le fuseau horaire, les profils et le nom des comptes.

— **Consigné** : chemin, SHA-256 et contenu du fichier vont dans `investigation.json` (`Tool.Configuration`) — la procédure appliquée fait partie de la preuve.

— Un artefact désactivé n'est pas lu du tout, et son fichier JSON le dit (`CollectionStatus NotRequested`, avec la clé) : jamais confondu avec un échec ou une absence de traces.

— `--config=fichier` lit un autre fichier.

— Sans `wac.yml`, WAC collecte tous les artefacts, journaux d'événements compris, options telles que données.

4.3 Options

Option	Effet
`--collect`	Le mode par défaut, dit explicitement : collecte, sur la machine examinée — relevés en direct et extraction brute dans la consigne scellée, rien n'est converti. Voir chapitre 5.
`--convert=dossier`	Conversion seule, sur un poste d'analyse, de la collecte faite avec `--collect` dans ce dossier, après vérification du sceau et de chaque empreinte. Voir chapitre 5.

Option	Effet
`--events`	Prend les journaux d'événements (`.evtx`) même si `wac.yml` dit `events: false` ; sans cela, ils sont pris par défaut (clé `events`). Chaque événement reçoit son message lisible, reconstitué à partir des ressources de son fournisseur. Environ 250 Mo de copies.
`--binary`	À la collecte, lit directement chaque exécutable, bibliothèque, pilote, script et document à macros des volumes fixes, vérifie sa signature en mémoire et **prélève** dans la consigne ceux qu'elle n'authentifie pas ; à la conversion, chaque fichier cité par un artefact reçoit les empreintes de sa pièce. Voir section 4.5.
`--binary-all`	Comme `--binary`, mais **tout** exécutable est prélevé, authentifié ou non ; la signature est tout de même vérifiée, et le manifeste dit pour chacun si elle est valide. Pour les dossiers où le contenu même doit figurer dans la consigne.
`--threads=N`	Nombre de fils qui analysent les exécutables de `--collect --binary` (PE, Authenticode, catalogues, empreintes). Par défaut, le nombre de fils du processeur moins un, la lecture du disque gardant le sien. Le résultat n'en dépend pas : le manifeste est le même.
`--update-trust [=dossier]`	Sur le **poste d'analyse**, **avant** la collecte, et seule : télécharge chez Microsoft la liste des racines de confiance (`authroot.stl`), celle des certificats désapprouvés (`disallowedcert.stl`) et chacune des racines, vérifie leurs signatures et leurs empreintes, ainsi que les listes de pilotes vulnérables de Microsoft et de LOLDrivers (non signées : authentifiées par HTTPS seul) et les listes de révocation (CRL) des autorités de signature de code que recense la CCADB, de celles propres à Microsoft et de celles qu'une collecte précédente a trouvées manquantes (`wanted-crls.txt`), et les écrit dans `dossier` (par défaut `trust`, à côté de `WAC.exe`), à emporter sur la clé avec `WAC.exe`. Seul mode qui utilise le réseau : la bibliothèque réseau n'est chargée que là. Refusée avec toute autre option.
`--dump`	Ajoute la valeur hexadécimale brute des éléments de shellbags et de raccourcis `.lnk`, pour un décodage ultérieur.
`--output=dossier`	Dossier de sortie, relatif au répertoire courant. Par défaut : `output`.
`--language=fr|en`	Langue de la console : étapes, aide, messages d'erreur. Français par défaut ; `language:` dans `wac.yml` la fixe aussi. Les fichiers JSON, `investigation.json` et le journal texte restent en anglais, quelle que soit la langue : ce sont des pièces lues par des outils, et une collecte donne les mêmes fichiers quel que soit l'opérateur.

Option	Effet
`--loglevel=N`	Active un journal texte détaillé, dans le répertoire courant, nommé d'après le programme lancé (`WAC.exe.log` pour `WAC.exe`) : 0 aucun, 1 par type d'artefact, 2 par artefact, 3 par sous-fonction (mise au point).
`--debug`	Affiche sur la sortie d'erreur la trace du lecteur NTFS (résolution des chemins, blocs d'index, séquences de grappes). Réservé au diagnostic.
`--help, /?`	Affiche l'aide.

4.4 Exemples

Collecte standard :

```
E:\> WAC.exe --output=poste-comptable-01
```

Collecte complète, avec journaux, binaires et journal détaillé :

```
E:\> WAC.exe --events --binary --output=poste-comptable-01 --loglevel=2
```

4.5 L'option `--binary`

Une empreinte suffit pour interroger une base publique (VirusTotal, par exemple) **sans lui envoyer le fichier**. Mais elle ne dit rien d'un binaire que personne ne connaît — précisément celui qui intéresse l'enquête — et un binaire non prélevé peut avoir disparu quand une détection survient. C'est pourquoi `--binary` copie aussi les charges potentielles :

— **prélevés** : `exe`, `dll`, `sys`, `ocx`, `cpl`, `scr`, `drv`, `efi`, `com`, `msi` et les scripts `ps1`, `psm1`, `bat`, `cmd`, `vbs`, `vbe`, `js`, `jse`, `wsf`, `wsh`, `hta` ;

— **prélevés aussi** : les documents Office **capables de porter des macros**, vecteur d'intrusion aussi courant qu'un exécutable — formats binaires anciens (`doc`, `xls`, `ppt`, `pps`, `dot`, `xlt`, `pot`), formats « m » (`docm`, `dotm`, `xlsm`, `xltm`, `pptm`, `potm`, `ppsm`), `xlsb`, compléments (`xla`, `xlam`, `xll`, `wll`, `ppa`, `ppam`), Publisher (`pub`), Visio (`vsd`, `vsdm`, `vstm`, `vssm`) et Access (`mdb`, `accdb`, `accde`). Ils apparaissent dans les traces comme cibles de raccourcis ou de listes de sauts, ou parmi les fichiers chargés par Word ou Excel dans leur Prefetch ;

— **seulement hachés** : les autres documents et fichiers de données, y compris `docx`, `xlsx` et `pptx`, qui ne peuvent pas contenir de macros. Les copier ferait de la collecte une copie des fichiers de l'utilisateur.

Les fichiers cités proviennent des processus, services (binaire et DLL), tâches planifiées,

fichiers chargés par les programmes (Prefetch), Shimcache, Amcache et cibles de raccourcis et de listes de sauts. Chaque fichier n'est lu qu'une fois, quel que soit le nombre d'artefacts qui le citent, et **un même contenu n'est stocké qu'une fois** : les autres chemins figurent au manifeste comme pièces partageant ce contenu (`SharedExhibit`).

Binaires Microsoft authentiques : hachés, non prélevés

La plupart des binaires cités sont des composants de Windows, identiques sur toute machine de la même version. Leur origine se prouve sur la machine elle-même, sans liste à embarquer : les **catalogues** de Windows, signés par Microsoft, listent l'empreinte de chaque fichier du système. WAC lit ceux de `System32\CatRoot`, ceux des composants (`WinSxS\Catalogs`) et ceux des paquets de maintenance (`servicing\Packages`), plus de 13 000 sur Windows 11. Les binaires signés individuellement (Edge, OneDrive, Office, .NET) portent une **signature intégrée**. Un binaire authentifié ainsi est haché sans être prélevé ; tous les autres sont prélevés — y compris un binaire système remplacé par un attaquant, dont l'empreinte ne correspond plus.

Cette vérification ne laisse **aucune trace** : catalogues et binaires sont lus directement sur le disque, et la vérification cryptographique est faite en mémoire par WAC, jusqu'à des certificats racines Microsoft embarqués dans l'exécutable. Aucun service n'est sollicité, le magasin de certificats de la machine n'est pas consulté, et la révocation n'est pas contrôlée en ligne.

Seuls les signataires Microsoft pour son propre code sont acceptés (« Microsoft Windows », « Microsoft Corporation », « Microsoft Windows Publisher » et « .NET »). Les pilotes **tiers** certifiés par Microsoft (signataires « Hardware Compatibility Publisher » et « Early Launch Anti-malware Publisher ») restent prélevés : un pilote vulnérable mais signé est un moyen d'attaque classique.

Applications du Store : vérifiées par leur paquet

Les fichiers d'une application du Store ne sont pas signés un à un : c'est leur **paquet** qui l'est. Son dossier contient la signature (`AppxSignature.p7x`) et la carte des blocs (`AppxBlockMap.xml`), qui donne le SHA-256 de chaque bloc de 64 Kio de chaque fichier. WAC vérifie la signature, qui doit remonter à une racine Microsoft (le Store signe les paquets qu'il distribue), vérifie qu'elle garantit bien cette carte, puis compare chaque bloc lu à la carte. Un fichier conforme est haché sans être prélevé ; un fichier modifié, ou ajouté au dossier après l'installation, est prélevé, de même que tout fichier d'un paquet signé hors de la chaîne Microsoft.

Les catalogues, signatures et cartes de blocs qui ont justifié un non-prélèvement sont consignés, pour qu'un tiers puisse refaire la vérification. Dans les fichiers d'artefacts, le champ `Signature` (ou `ServiceDllSignature`, `SignatureTarget`…) indique l'authentification retenue, par exemple « `Microsoft (catalog C:\Windows\….cat)` », « `Microsoft (embedded signature)` » ou « `Package (package C:\Program Files\WindowsApps\…)` ».

Les **scripts** sont authentifiés de la même façon : par les catalogues de Windows, et, pour PowerShell (`ps1`, `psm1`, `psd1`, `ps1xml`…), par leur bloc de signature intégré (`# SIG # Begin signature block`). Un script Windows Script Host (`vbs`, `js`, `wsf`) signé seulement dans son

texte, et non par un catalogue, reste prélevé.

Sur la machine de test, cette vérification ramène le prélèvement de 2 131 binaires (3,3 Go) à 92 (335 Mo), sans allonger la collecte ; les scripts encore prélevés sont ceux qui ne sont pas signés.

La lecture étant directe, prélever ne laisse aucune trace de plus que hacher : seulement de la place sur le support.

4.6 Déroulement à l'écran

WAC affiche chaque étape, suivie de `OK` ou d'un message d'erreur. Une étape en échec n'interrompt pas la collecte : tout ce qui est accessible est recueilli, et ce qui manque est consigné.

```
[VÉRIFICATION DES PRÉREQUIS]
 - Windows 10 ou ultérieur : OK
 - Droits d'administrateur : OK
 - Contrôle du support de collecte : OK
[OBSERVATION DE LA MACHINE EN MARCHE]
 - Sessions : OK
 - Processus : OK
 - État du réseau : OK
[EXTRACTION BRUTE]
 - Ruches du système (lecture brute NTFS) : OK
 - Ouverture de la ruche SYSTEM : OK
 - Ouverture de la ruche SOFTWARE : OK
 - Profils des utilisateurs (ruche SOFTWARE) : OK
 - Ruches des utilisateurs (lecture brute NTFS) : OK
 - Artefacts fichiers (lecture brute NTFS) : OK
[ARTEFACTS DU REGISTRE]
 - Clés UserAssist du registre : OK
   ...
[ARTEFACTS DES FICHIERS]
 - Prefetch : OK
   ...
[JOURNAUX D'ÉVÉNEMENTS]
 - Journaux d'événements : OK
[CONSIGNE]
 - Copie des binaires prélevés dans la copie de travail : OK
 - Scellement de la consigne (manifeste + SHA-256) : OK
[JOURNAL D'INVESTIGATION]
 - Écriture de investigation.json : OK
FIN, durée : 309 s
```

Avec `--language=en`, ou `language: en` dans `wac.yml`, les mêmes étapes s'affichent en anglais.

Le scellement de la consigne est la **dernière** opération sur celle-ci : toutes les pièces, y compris celles ajoutées pendant l'analyse des journaux, sont couvertes par le manifeste.

4.7 Durée

La durée dépend surtout du matériel (USB 2 ou 3, disque, processeur) et des options. Sur la machine virtuelle de test (disque NVMe) : environ 70 s avec `--events`, environ 310 s avec `--events` et `--binary`. Sur une clé USB, l'écriture domine.

CHAPITRE 5

Collecter ici, convertir ailleurs

5.1 Pourquoi séparer

Une exécution de WAC fait deux choses différentes :

1. **la collecte** : relever ce que seul un système en marche montre (processus, sessions, état des services, horloge), lire le disque directement, copier les preuves dans une consigne scellée. Elle a **besoin** de la machine examinée ;
2. **la conversion** : décoder en JSON des milliers de fichiers copiés (ruches, journaux, Prefetch, raccourcis…). Elle n'a besoin que de la consigne, **pas** de la machine.

WAC ne fait **jamais les deux dans la même exécution** : il collecte sur la machine examinée, et convertit ailleurs. Convertir sur la machine y ajouterait un travail que rien n'impose : des minutes de traitement dans la mémoire de WAC, que Windows peut paginer dans `pagefile.sys`, sur le disque même qu'on examine ; un processus présent plus longtemps, vu et journalisé par les protections de la machine ; en cas de plantage, un rapport d'erreur Windows écrit sur la machine ; et des bibliothèques du système chargées pour décoder (le moteur ESE pour rejouer les journaux d'une base).

C'est précisément ce que les bonnes pratiques demandent d'éviter :

— **ISO/IEC 27037** traite de l'identification, de la collecte et de la préservation des preuves numériques, **ISO/IEC 27042** de leur analyse : deux étapes distinctes ;

— **principe 1 de l'ACPO** : aucune action ne doit modifier les données de la machine examinée ; toute opération qui s'y exécute doit être nécessaire ;

— **principe 3 de l'ACPO** : un tiers indépendant doit pouvoir refaire le traitement et obtenir le même résultat.

Séparer les deux étapes apporte trois choses :

— **moins de traces** sur la machine examinée : seule la collecte s'y exécute ;

— **des preuves vérifiées avant toute analyse** : `--convert` contrôle d'abord le sceau et le SHA-256 de chaque pièce, et refuse une collecte retouchée ;

— **une analyse reproductible** : deux conversions d'une même collecte donnent des fichiers JSON identiques octet pour octet, et l'analyse peut être refaite plus tard, avec une version corrigée de WAC ou par un tiers, sans revenir sur la machine.

5.2 Les réglages de la collecte

— **Portée de `--binary`** : la collecte lit **tous les exécutables de tous les volumes fixes NTFS**, cités ou non, car elle ne sait pas encore lesquels les artefacts citeront. Ce temps (8 min 30 sur la machine de test) est de la lecture, qui n'écrit rien sur la machine.

— **`--binary-all`** : quand le contenu de chaque exécutable doit figurer dans la consigne (analyse de logiciel malveillant, demande des fichiers eux-mêmes), pour environ 20 Go.

5.3 Les deux commandes

Sur la machine examinée, en administrateur :

```
E:\> WAC.exe --collect --events --binary --output=poste-comptable-01
```

Sur le poste d'analyse, sans droits particuliers, avec la même configuration (`events`, `--binary`) que la collecte :

```
D:\> WAC.exe --convert=E:\poste-comptable-01 --events --binary
```

Les fichiers JSON sont écrits dans le dossier de la collecte, à côté de la consigne. `--convert` exclut `--collect` et `--output`.

5.4 Ce que prend `--collect`

La conversion ne pouvant plus rien demander à la machine, la collecte prend **tout ce qu'elle lira** :

— les **relevés en direct**, enregistrés tels que lus dans `exhibits\live` et scellés comme toute pièce : processus, sessions, état des services, horloge et démarrage, et les réglages de la machine (lecteur système, page de code, fuseau horaire, volumes et leurs numéros de série) ;

— les ruches et les fichiers d'artefacts, comme d'ordinaire ;

— avec les journaux d'événements (`events`), les fichiers de ressources de **tous** les fournisseurs d'événements déclarés (binaire, fichier de paramètres, satellites `.mui`), car on ne sait qu'à la conversion lesquels seront utiles. Cela représente environ 650 Mo sur Windows 11 ;

— avec `--binary`, **tous** les exécutables, bibliothèques, pilotes, scripts et documents à macros de **tous les volumes fixes NTFS** — un profil sur `D:` compris —, lus directement et vérifiés en mémoire (section 4.5) :

  — authentifié (catalogue, signature intégrée, paquet du Store), ou binaire tiers dont la signature tient face au jeu de confiance de la clé (`--update-trust` : chaîne vers une racine admise par Microsoft pour la signature de code, valide à la date

de signature, non révoquée, pas un pilote vulnérable) : **non copié** ; sans jeu de confiance utilisable, tout binaire tiers est copié, et le journal d'investigation le dit ; le manifeste garde son chemin, ses empreintes Authenticode et le verdict (`ContentStored` à `false`) ;

— non authentifié : **copié**, avec ses trois empreintes MD5, SHA-1, SHA-256, son empreinte Authenticode et la raison ;

— fichier différentiel de Windows (`PA30`/`PA31`, dossiers `r` et `f` de `WinSxS`) : ce n'est pas un exécutable ; haché, non copié.

Un même contenu n'est stocké qu'une fois, et les autres noms d'un lien physique (`System32` et `WinSxS`) sont déclarés au manifeste comme le premier. Les volumes amovibles, dont le support de collecte, et les fichiers de mémoire (`pagefile.sys`, `swapfile.sys`, `hiberfil.sys`) sont exclus.

> **Attention**
> Avec `--binary`, la collecte lit tous les volumes : sur la machine virtuelle de test (Windows 11 sans logiciel ajouté), 44 446 exécutables lus, dont 41 593 authentifiés (ou tiers à signature valide) non copiés et 2 853 copiés (1,2 Go) ; la collecte entière dure 8 minutes 30 et la consigne, journaux d'événements et métadonnées NTFS compris, pèse 2,7 Go (voir la section 3.2). Si la place vient à manquer, les fichiers restants sont hachés sans être copiés, et le manifeste l'indique.

Aucun fichier JSON d'artefact n'est écrit : le dossier ne contient que la consigne scellée, le répertoire de travail et `investigation.json`.

5.5 Ce que fait `--convert`

1. **Vérification**, avant toute lecture : le sceau doit porter l'empreinte réelle du manifeste, et chaque pièce son empreinte SHA-256. Une seule pièce altérée, et la conversion est **refusée** ; `conversion.json` en donne la raison.
2. **Reconstruction du répertoire de travail** depuis la consigne : l'ancien est supprimé plutôt que cru, les copies sont vérifiées contre le manifeste et les ruches réparées, comme à la collecte. Les exécutables et les ressources des fournisseurs sont lus dans la consigne même, sans copie.
3. **Conversion**, depuis les seules pièces : aucune lecture de la machine qui convertit, ni de son disque, ni de son registre, ni de son fuseau horaire.

La consigne n'est jamais modifiée et son sceau reste celui de la collecte. `investigation.json` reste le journal de la collecte ; `conversion.json` est celui de la conversion (opérateur, poste, opérations). Deux conversions d'une même collecte donnent des fichiers JSON identiques, ce que le harnais de test vérifie à chaque cycle.

5.6 Limites

— Les **documents à macros** (`doc`, `docm`, `xls`, `xlsm`, `ppt`…, liste de la section 4.5) sont traités comme les exécutables : lus sur tous les volumes fixes et **prélevés**, sauf s'ils sont, octet pour octet, un fichier livré par Microsoft et listé dans un catalogue — ce qu'un document écrit ou modifié par quiconque ne peut pas être.

— Un fichier cité qui ne peut porter ni code ni macro (`docx`, `xlsx`, `pdf`, donnée) n'est pas prélevé : en conversion, il n'a donc **pas d'empreinte**.

— Un exécutable authentifié n'est pas copié : à la conversion, il a son empreinte Authenticode et son verdict, mais pas de MD5, SHA-1 ni SHA-256 — ceux-ci ne sont calculés que pour ce qui est prélevé.

— Un exécutable situé sur un volume amovible n'est pas prélevé par `--collect --binary`.

— La configuration (`events`, `--binary`) de la conversion doit être celle de la collecte : ce que la collecte n'a pas pris ne peut pas être converti.

CHAPITRE 6

Les résultats

6.1 Arborescence

```
<sortie>/
  exhibits/            copies brutes, jamais réécrites
    MANIFEST.json     identification de chaque pièce et de la collecte
    MANIFEST.sha256   empreinte du manifeste : son sceau
    Windows/...        les pièces, sous leur chemin d'origine
    Users/...
  working/             copies de travail (ruches rejouées)
  investigation.json   journal de la collecte
  *.json               les artefacts
```

Un fichier d'un autre volume que le volume système est rangé sous `_volume_D\...` ; le JSON indique toujours le chemin d'origine, avec sa vraie lettre de lecteur.

6.2 Le manifeste de consigne

`exhibits/MANIFEST.json` identifie la collecte et chaque pièce.

6.2.1 En-tête

`Tool`
: nom de l'outil, ligne de commande, date de compilation.

`Host`
: nom de la machine, lecteur système, fuseau horaire du suspect (lu dans sa ruche `SYSTEM`) et de la machine de collecte.

`Operator`
: compte ayant lancé la collecte, et s'il était élevé.

`Custody`
: déclaration de procédure, début de l'extraction, heure d'écriture du manifeste, nombre de pièces et d'échecs, volume total, algorithmes d'empreinte, volumes lus (lettre, numéro de série, système de fichiers).

6.2.2 Chaque pièce

Champ	Signification
`SourcePath`	Chemin d'origine, avec lettre de lecteur.
`ExhibitPath`	Emplacement de la copie brute, relatif au dossier de sortie.
`Method`	Méthode de collecte (lecture directe NTFS, volume, motif).
`Result`	`OK`, ou le code et le message d'erreur. Les échecs figurent au manifeste : une pièce absente se lirait comme une pièce jamais cherchée.
`MD5, SHA1, SHA256`	Les trois empreintes, calculées pendant l'écriture : elles portent sur ce qui a été lu du volume.
`Bytes`	Taille extraite.
`DeclaredBytes`	Taille déclarée par NTFS, présente seulement si elle diffère de la taille extraite (extraction tronquée).
`ValidDataBytes`	Longueur de données valides d'un fichier préalloué (journaux d'événements) : au-delà, la pièce contient des zéros par définition.
`SharedExhibit`	`true` si le contenu, identique octet pour octet, est déjà stocké sous une autre pièce : `ExhibitPath` désigne celle-ci. La source reste une pièce à part entière, avec ses propres chemin, entrée `$MFT` et horodatages.
`MftEntry`	Numéro d'enregistrement NTFS : identifie le fichier indépendamment de son nom.
`Extracted(Utc)`	Instant de l'extraction de cette pièce.
`SourceCreated, SourceModified, SourceMftModified, SourceAccessed`	Les quatre horodatages NTFS du fichier d'origine (`$STANDARD_INFORMATION`), en UTC et en heure locale du suspect. Ce sont eux, et non les dates des copies, qui datent le fichier.
`SourceFileName Created, Modified, MftModified, Accessed`	Les quatre horodatages de l'attribut `$FILE_NAME` du nom lu. NTFS les tient à part, et une modification des dates par programme ne les touche pas : un `SourceCreated` antérieur à `SourceFileNameCreated` est le signe classique de dates falsifiées après coup (*timestomping*).
`ContentStored`	`false` : binaire authentifié lu mais non copié (`--collect --binary`) ; il n'a pas de `ExhibitPath`.
`AuthenticodeSHA1, AuthenticodeSHA256`	Empreintes Authenticode d'un PE, celles que couvrent sa signature et les catalogues ; seules empreintes d'un binaire authentifié non copié.
`SignatureVerified`	Résultat de la vérification de signature d'un exécutable : `true` avec `Signature` (ce qui l'authentifie), `false` avec `SignatureReason` (pourquoi).

Champ	Signification
`PeTimeDateStamp`, `PeSizeOfImage`, `SymbolServerKey`	Version d'un PE et clé (`nom/HORODATAGEtaille/nom`) sous laquelle le serveur de symboles de Microsoft la conserve : un binaire Microsoft authentique non copié peut en être retéléchargé, identique, et son empreinte revérifiée.

> **Pourquoi trois empreintes**
> Des collisions MD5 se fabriquent à volonté depuis 2008, et SHA-1 depuis 2017 : une empreinte seule ne tranche plus une contestation d'identité. Trois empreintes indépendantes, si.

6.3 Vérifier l'intégrité d'une collecte

6.3.1 Le sceau

Le manifeste ne peut pas contenir sa propre empreinte : elle est dans `MANIFEST.sha256`. Sous Linux :

```
$ cd sortie/exhibits
$ sha256sum -c MANIFEST.sha256
MANIFEST.json: OK
```

6.3.2 Chaque pièce

Deux scripts, livrés avec `WAC.exe` dans `bin`, vérifient le sceau puis le SHA-256 de chaque pièce : `verifier-consigne.ps1` sous Windows, `verifier-consigne.sh` sous Linux. Leurs équivalents anglais, `verify-exhibits.ps1` et `verify-exhibits.sh`, font les mêmes contrôles avec des messages en anglais.

Vérifier depuis une machine Linux. C'est la voie recommandée : la collecte peut contenir ce qui a infecté la machine examinée, et une machine d'investigation sous Linux constitue une rupture technologique — un exécutable ou un document à macro Windows de la collecte ne peut pas s'y exécuter. Le script ne demande que `bash`, les coreutils et `python3` (qui lit le JSON) ; les empreintes sont calculées par `sha256sum`, non par `WAC` :

```
$ bash verifier-consigne.sh /media/usb/output
Sceau conforme (DCFF53FAE709EC2E351FFFD02EC6EAD9945602265CF3363D0D928B0B452C3E32)
3360 piece(s) conforme(s), 0 differente(s)
0 binaire(s) authentifie(s), empreinte seule (pas de copie a verifier)
```

Le manifeste vient de la machine examinée : les deux scripts refusent un chemin de pièce qui sort de la collecte (`..`, chemin absolu, et pour le `.sh` un lien symbolique), et le comptent comme différent sans le lire.

Depuis Windows :

```
PS> powershell -ExecutionPolicy Bypass -File verifier-consigne.ps1 -Sortie E:\output
Sceau conforme (DCFF53FAE709EC2E351FFFD02EC6EAD9945602265CF3363D0D928B0B452C3E32)
3360 piece(s) conforme(s), 0 differente(s)
```

Leur code de retour vaut 0 si tout est conforme, 1 sinon.

```
#!/usr/bin/env bash
# verifier-consigne.sh — checks the seal and every exhibit of a WAC collection
# (French output messages; same checks as verify-exhibits.sh),
# from a Linux investigation machine: the collection is examined away from the
# Windows machine it comes from, which it cannot infect there.
# Usage: bash verifier-consigne.sh /media/usb/output
# Exit code: 0 if everything matches, 1 otherwise.
# Needs: bash, coreutils (sha256sum, realpath), python3 (reads the JSON only).
#
# The fingerprints are computed by sha256sum, not by WAC: a verification that
# does not rely on the tool that produced the collection.
#
# The manifest comes from the examined machine, so it is treated as hostile:
# an exhibit path that leaves the collection (.., absolute path, symbolic
# link) is refused and counted as a mismatch, never read.

set -u
sortie="${1:-./output}"
manifest="$sortie/exhibits/MANIFEST.json"
seal="$sortie/exhibits/MANIFEST.sha256"

for tool in sha256sum realpath python3; do
    command -v "$tool" >/dev/null || { echo "OUTIL ABSENT : $tool"; exit 1; }
done
[ -f "$manifest" ] && [ -f "$seal" ] || { echo "MANIFESTE ABSENT : $manifest ou $seal";
    exit 1; }
root="$(realpath -e -- "$sortie")" || exit 1

# 1. The seal: the manifest's fingerprint must be the one recorded beside it.
recorded="$(head -c 64 -- "$seal" | tr 'a-f' 'A-F')"
actual="$(sha256sum -- "$manifest" | head -c 64 | tr 'a-f' 'A-F')"
if [ "$recorded" != "$actual" ]; then echo "SCEAU NON CONFORME : manifeste modifie"; exit
    1; fi
echo "Sceau conforme ($actual)"

# 2. Every exhibit: its SHA-256 must be the one in the manifest.
# python3 lists the exhibits kept with their content, as NUL-separated
# "S, SHA-256, path" records, and one "F" record per authenticated binary that
# was not copied (--collect --binary): no file to verify. NUL cannot occur in
# a path, whatever the manifest holds.
list_exhibits() {
    python3 - "$manifest" <<'PYTHON'
import json, sys
with open(sys.argv[1], encoding="utf-8-sig") as f:
    items = json.load(f).get("Items") or []
out = sys.stdout.buffer
```

```
for item in items:
    if not isinstance(item, dict) or item.get("Result") != "OK":
        continue
    if item.get("ContentStored") is False:
        out.write(b"F\0")
        continue
    sha = str(item.get("SHA256") or "").replace("\0", "")
    path = str(item.get("ExhibitPath") or "").replace("\0", "")
    out.write(b"S\0" + sha.encode() + b"\0" + path.encode("utf-8", "surrogateescape") + b
    "\0")
PYTHON
}

ok=0; ko=0; fingerprint_only=0
while IFS= read -r -d '' kind; do
    if [ "$kind" = F ]; then fingerprint_only=$((fingerprint_only + 1)); continue; fi
    IFS= read -r -d '' expected || break
    IFS= read -r -d '' exhibit || break
    # The manifest writes Windows paths, relative to the output folder.
    relative="${exhibit//\\//}"
    file="$(realpath -e -- "$root/$relative" 2>/dev/null)" || file=""
    case "$relative" in
        ""|/*) file="" ;;
    esac
    case "$file" in
        "$root"/*) ;;
        *) ko=$((ko + 1)); echo "DIFFERENTE : $exhibit"; continue ;;
    esac
    hash="$(sha256sum -- "$file" | head -c 64 | tr 'a-f' 'A-F')"
    if [ -n "$expected" ] && [ "$hash" = "$(printf '%s' "$expected" | tr 'a-f' 'A-F')" ];
    then
        ok=$((ok + 1))
    else
        ko=$((ko + 1)); echo "DIFFERENTE : $exhibit"
    fi
done < <(list_exhibits || printf 'S\0\0unreadable manifest\0')

echo "$ok piece(s) conforme(s), $ko differente(s)"
echo "$fingerprint_only binaire(s) authentifie(s), empreinte seule (pas de copie a
    verifier)"
[ "$ko" -eq 0 ] || exit 1
```

```
# verifier-consigne.ps1 — checks the seal and every exhibit of a WAC collection
# (French output messages.) Usage: powershell -ExecutionPolicy Bypass -File verifier-
    consigne.ps1 -Sortie E:\output
# Exit code: 0 if everything matches, 1 otherwise.

param([string]$Sortie = '.\output')

# 1. The seal: the manifest's fingerprint must be the one recorded beside it.
$annonce = ((Get-Content -Raw "$Sortie\exhibits\MANIFEST.sha256") -split '\s+')[0]
$reel    = (Get-FileHash -Algorithm SHA256 "$Sortie\exhibits\MANIFEST.json").Hash
if ($annonce -ne $reel) { Write-Output "SCEAU NON CONFORME : manifeste modifie"; exit 1 }
```

```
Write-Output "Sceau conforme ($reel)"

# 2. Every exhibit: its SHA-256 must be the one in the manifest.
$m = Get-Content -Raw -Encoding UTF8 "$Sortie\exhibits\MANIFEST.json" | ConvertFrom-Json
# The manifest comes from the examined machine: an exhibit path that leaves
# the collection (.., absolute path) is refused, never read.
$root = (Resolve-Path -LiteralPath $Sortie).ProviderPath.TrimEnd('\') + '\'
$ok = 0; $ko = 0; $fingerprintOnly = 0
foreach ($p in ($m.Items | Where-Object { $_.Result -eq 'OK' })) {
    # An authenticated binary that was not copied (--collect --binary): no file.
    if ($p.ContentStored -eq $false) { $fingerprintOnly++; continue }
    try { $f = [IO.Path]::GetFullPath((Join-Path $root $p.ExhibitPath)) } catch { $f = ''
    }
    $h = $null
    if ($f.StartsWith($root, [StringComparison]::OrdinalIgnoreCase)) {
        $h = (Get-FileHash -Algorithm SHA256 -LiteralPath $f -ErrorAction SilentlyContinue
    ).Hash
    }
    if ($h -and $h -eq $p.SHA256) { $ok++ } else { $ko++; Write-Output "DIFFERENTE : $($p.
    ExhibitPath)" }
}
Write-Output "$ok piece(s) conforme(s), $ko differente(s)"
Write-Output "$fingerprintOnly binaire(s) authentifie(s), empreinte seule (pas de copie a
    verifier)"
if ($ko) { exit 1 }
```

6.4 Le journal d’investigation

`investigation.json` retrace la collecte : outil et ligne de commande, machine et fuseaux horaires, opérateur, début, fin et durée, puis la liste **ordonnée** des opérations. Pour chacune :

`Seq`, `TimestampUtc`, `TimestampLocal`
: rang et instant ;

`Operation`, `Target`
: ce qui a été fait, et sur quoi ;

`Result`
: l’issue ;

`Footprint`
: la trace que cette opération est censée laisser sur la machine examinée.

Ce journal permet de confronter, collecte par collecte, les traces attendues à celles effectivement observées. Il signale aussi un écart entre le fuseau horaire du suspect et celui de la machine de collecte.

6.5 Les fichiers d'artefacts

Un fichier par artefact. Les champs décrits sont ceux émis ; un champ sans valeur n'est pas émis (voir chapitre 7).

6.5.1 Système et comptes

`OperatingSystem.json`
nom et domaine de la machine, édition et version de Windows, date d'installation, heure courante, **heure de démarrage**, durée d'activité, fuseau horaire. `BootTimeSource` dit d'où vient l'heure de démarrage (noyau, ou estimation de repli) ; `ClockAdjustedSinceBootMs` donne le cumul des corrections d'horloge depuis le démarrage — quelques secondes sont un recalage ordinaire, un grand écart signale une horloge modifiée. `ShutdownTime` : le dernier arrêt ordonné. `NtfsLastAccessUpdated` : faux, les dates d'accès NTFS ne sont pas mises à jour et ne disent rien. `Settings` : les réglages qui conditionnent la lecture des autres artefacts, tels que stockés — dates d'accès et noms courts NTFS, Prefetch actif, fichier d'échange et vidages mémoire, jeux de contrôle, bureau à distance, chronologie Windows, dernier utilisateur connecté.

`security.json`
ruche `SECURITY` : **noms** des secrets LSA et dates de leurs valeurs (`DefaultPassword` signale un mot de passe de connexion automatique enregistré, `_SC_<service>` un service qui tourne sous un compte), nombre d'emplacements de connexions de domaine en cache et combien sont occupés, politique d'audit (`PolAdtEv`, en octets : son format n'est pas publié), et les droits des comptes (`AccountRights`, lus dans `Policy\Accounts`) : pour chaque compte ou groupe, ses privilèges (`Privileges`, p. ex. `SeDebugPrivilege`, `SeBackupPrivilege`) et ses droits de connexion (`LogonRights`). Un privilège sensible donné à un compte inattendu est une persistance. **Aucune valeur de secret** n'est lue, ni aucune donnée d'authentification en cache. Les copies `config\RegBack` des ruches sont prélevées telles quelles.

`users.json`
comptes locaux, lus dans la ruche `SAM` : SID et RID, état, drapeaux, nombre de connexions et d'échecs, dates de dernière connexion, de dernier échec, de changement de mot de passe et de modification du compte, profil, et les condensats NT et LM du mot de passe (`NtHash`, `LmHash`, en hexadécimal), avec leur historique (`NtHashHistory`, `LmHashHistory`). Un condensat est à sens unique : il désigne un mot de passe sans le révéler. Un mot de passe vide ou faible utilisé par un intrus se lit comme une connexion légitime ; **WAC ne juge rien**, il convertit, et c'est WAE qui confronte les condensats, hors ligne, à ceux des mots de passe faibles connus. Les condensats sont déchiffrés sur place (clé de démarrage de `SYSTEM`, clé de hachage de la `SAM`, schéma AES de Windows 10 et suivants ou RC4 plus ancien, puis couche DES par RID). Si le compte porte des questions de sécurité de réinitialisation (`SecurityQuestions`, valeur `ResetData`), elles sont converties telles quelles, questions ET réponses : stockées en clair, une réponse triviale est une faiblesse que WAE juge comme un mot de passe faible.

`local_groups.json`
groupes locaux lus dans la `SAM` : groupes intégrés (`Scope Builtin`, `S-1-5-32-…`) et groupes

créés sur la machine (`Local`), chacun avec son RID, son SID, son nom, sa description et ses membres directs (SID et nom résolu hors ligne). C'est l'appartenance qui dit qui est administrateur ou membre des Bureaux à distance ; un compte ajouté à un groupe privilégié par un intrus y figure.

`Sessions.json`
sessions ouvertes au moment de la collecte : utilisateur, type d'ouverture, paquet d'authentification, heure de début.

`processes.json`
processus en cours : nom, identifiants (et parent), propriétaire, session, modules chargés, et, avec `--binary`, empreintes de l'exécutable.

`network.json`
état réseau au moment de la collecte : `Connections` (TCP et UDP, IPv4 et IPv6 : adresses, ports, état, processus propriétaire et son nom, heure de création locale et UTC), `Neighbours` (caches ARP et NDP : les machines du réseau local contactées récemment), `Interfaces` (adresses, passerelles, serveurs DNS, DHCP) et `Routes`. `TablesNotRead` nomme une table que Windows a refusé de donner, ce qui n'est pas une table vide.

`services.json`
services et pilotes, configuration lue dans la ruche : type, démarrage, compte, binaire et DLL hébergée, commande en cas d'échec, dépendances, dernière écriture de la clé ; état courant ; avec `--binary`, empreintes du binaire et de la DLL.

`ScheduledTasks.json`
tâches planifiées, lues dans leurs définitions XML : auteur, compte d'exécution, état, dernière exécution et son résultat, déclencheurs et actions (avec `--binary`, empreintes de la commande).

6.5.2 Exécution de programmes

`prefetchs.json`
fichiers Prefetch : programme, nombre et dates d'exécution, volumes, et liste des fichiers chargés avec leur référence `$MFT` (et leurs empreintes avec `--binary`). Une référence absente n'est pas une lecture ratée : Windows l'écrit à zéro, pour tous les fichiers d'un programme lancé une seule fois et pour une partie des autres.

`amcache_application_files.json`, `amcache_applications.json`, `amcache_inventory.json`
inventaire Amcache, table par table et valeur par valeur, avec la date de dernière écriture de chaque entrée : les exécutables (chemin, éditeur, version, taille, date d'édition de liens, `Sha1`), les applications installées, et dans le troisième fichier les pilotes (avec leur `Sha1`), périphériques, conteneurs, paquets de pilotes et raccourcis, chacun avec sa `Table`. **Attention** : pour un fichier de plus de 30 Mio, Amcache ne hache que les 31 457 280 premiers octets ; `Sha1CoversBytes` le signale, et ce SHA-1 ne se compare pas à celui du fichier entier.

`shimcache.json`
cache de compatibilité : chemin et date de modification du fichier, position dans le cache

(`CachePosition`, 0 pour la plus récente). Une entrée prouve que le binaire était présent, pas qu'il a été exécuté. L'indicateur d'exécution que d'autres outils publient n'est *pas* relevé : mesuré contre le Prefetch, son « oui » est presque toujours juste, mais son « non » est faux pour 9 binaires exécutés sur 10 — il ferait conclure qu'un programme n'a jamais tourné.

`bams.json`
dernière exécution par programme et par utilisateur (Background Activity Moderator). Les ruches étant lues sur le disque, une exécution de la ou des dernières minutes peut manquer : Windows garde une modification du registre en mémoire avant de l'écrire (de 56 à 82 s mesurées sur la machine de test, que `RegFlushKey` n'a pas avancées), et `WAC` ne force pas cette écriture, qui modifierait le disque examiné.

`userassists.json`
programmes lancés depuis l'interface : compteur, nombre et durée de premier plan (`FocusTimeMs`), dernière exécution.

`muicache.json`
noms d'affichage des programmes exécutés.

`run.json`
programmes lancés au démarrage ou à l'ouverture de session (clés Run et RunOnce), avec la dernière écriture de la clé : dans SOFTWARE (toute la machine), dans la ruche DEFAULT (le compte système) et dans le NTUSER.DAT de chaque profil (`Hive`). `Type REG_EXPAND_SZ` : la commande contient des variables (`%SystemRoot%`…), données telles qu'écrites. Aussi : les clés Run des programmes 32 bits (`SOFTWARE (32-bit)`); `StartupApproved` (`Key StartupApproved\Run`, `Run32`, `StartupFolder`), l'état d'une entrée tel que l'utilisateur l'a réglé dans le Gestionnaire des tâches ou les Paramètres : `Status`, les 4 premiers octets tels que stockés (leur sens n'est pas mesuré, donc pas interprété), et `Changed`, l'instant du changement; `RunNotification`, une entrée de démarrage que Windows a remarquée (`Notification`). Un nom sans valeur Run correspondante est une entrée retirée depuis.

`persistence.json`
persistance du registre au-delà des clés Run : IFEO (`Debugger`, `GlobalFlag`), SilentProcessExit, AppInit_DLLs et AppCertDlls, Winlogon (`Shell`, `Userinit`, `Taskman`, `Notify`), paquets LSA, moniteurs d'impression, `BootExecute`, Active Setup, assistants NetSh, commandes d'ouverture des types exécutables (`exefile`, `batfile`, `cmdfile`…, machine et utilisateur), et détournements COM des classes de chaque utilisateur (UsrClass.dat). Pour chaque valeur : mécanisme (`Mechanism`), ruche, clé, nom, type, valeur telle qu'écrite, dernière écriture de la clé, et avec `--binary` les empreintes du fichier visé. Toutes les valeurs sont données, celles de Windows comprises : distinguer l'ajouté du défaut revient à l'analyse. Les tâches BITS ne sont pas encore lues.

`office_trust_records.json`
documents Office qu'un utilisateur a approuvés (« Activer le contenu ») : application, version, document, date; `MacrosEnabled` quand il a activé les macros — la porte de la plupart des infections par macro.

`typed_paths.json`, `word_wheel_query.json`
chemins tapés dans la barre d'adresse de l'Explorateur et termes qui y ont été

cherchés, par utilisateur et par position (`MruPosition`, 0 = le plus récent, comme dans `shellbags.json`).

`root_certificates.json`
certificats des magasins racines de la machine et de chaque utilisateur : sujet, émetteur, validité ; une racine ajoutée fait accepter tout ce qu'elle signe (interception TLS, logiciel signé). `ThumbprintMismatch` signale un certificat dont le SHA-1 n'est pas le nom de sa clé.

`internet_settings.json`
proxy et script de configuration automatique de chaque utilisateur.

`firewall.json`
règles du pare-feu (nom, action, sens, ports, programme ; tous les champs dans `Fields`) et réglages des profils domaine, privé et public.

`installed_programs.json`
programmes installés, lus dans les clés `Uninstall` de la machine (64 et 32 bits) et de chaque utilisateur, et produits Windows Installer (`Scope` `MSI product`, avec le paquet mis en cache `LocalPackage`) : toutes leurs valeurs (nom, éditeur, version, dossier, commande de désinstallation, `InstallDate` — le jour seul, tel qu'écrit) et la date d'écriture de la clé. Un outil d'accès à distance installé, puis désinstallé, y laisse souvent sa trace.

`defender_configuration.json`
configuration de Microsoft Defender lue dans le registre : les **exclusions** (chemins, extensions, processus, adresses ; `Source` `Settings` ou `Policy`), qu'un intrus ajoute pour que ses outils ne soient pas analysés, et les valeurs qui règlent ou coupent la protection (temps réel, protection contre la falsification, envoi d'échantillons…).

`explorer_recent_docs.json`
fichiers ouverts par chaque utilisateur, tels que l'Explorateur les retient (`RecentDocs`) : par extension (`Extension`) et pour toutes extensions confondues, avec leur rang (`MruPosition`, 0 pour le plus récent), l'élément de shell du raccourci créé dans Recent, et la date d'écriture de la liste, qui est celle de l'ouverture du fichier en position 0. Le nom survit au fichier et au raccourci.

`mount_points.json`
volumes et partages réseau qu'a vus l'Explorateur de chaque utilisateur (`MountPoints2`) : volume (GUID, à rapprocher de `mounted_device.json` pour une clé USB) ou partage (`Share`, `\\serveur\partage`), date d'écriture de l'entrée — la dernière fois que l'Explorateur l'a vue —, ses valeurs, et ses sous-clés avec leur commande (`Shell\AutoRun\command` : une persistance ancienne).

`known_networks.json`
réseaux auxquels la machine s'est connectée (`NetworkList`) : nom, type, catégorie, adresse matérielle de la passerelle et suffixe DNS de chaque signature, première et dernière connexion. Ces deux dates sont écrites par Windows en heure locale : WAC les donne en UTC avec le décalage en vigueur à leur date (mesuré contre le journal `NetworkProfile`). Elles disent où la machine a été.

`program_usage.json`
ce que chaque utilisateur a lancé ou utilisé, source par source (`Source`) : `FeatureUsage`

(lancements, bascules, clics sur la barre des tâches, par application, `Count`), `JumplistData` (dernière utilisation, `LastUsed`), le magasin de l'Assistant de compatibilité des programmes (programmes exécutés), les modes de compatibilité (`Layers`), les programmes diagnostiqués par RADAR, les éléments épinglés à la barre des tâches (`Taskband`), chacun décodé comme un raccourci complet, et les programmes choisis pour ouvrir chaque type de fichier (`FileExts` : `OpenWithList` avec `Extension` et `MruPosition`, 0 pour le plus récent ; `UserChoice`, le programme par défaut). `OpenWithProgids` n'est pas relevé : les installeurs le remplissent, l'utilisateur n'y est pour rien.

`devices.json`
: périphériques que la machine a connus (`Source`) : USB et SCSI (`Enum`), avec leurs valeurs et les dates d'installation, de première installation, de dernière arrivée et de dernier retrait (`Install`, `FirstInstall`, `LastArrival`, `LastRemoval`, en UTC) ; périphériques portables (téléphone, appareil photo) ; noms de volume que la recherche Windows a retenus par lettre de lecteur (`VolumeInfoCache`) ; interfaces de disque, de volume et de périphérique portable (`DeviceClasses`), écrites à leur première apparition.

`network_shares.json`
: dossiers que la machine partage (`LanmanServer\Shares`) : nom, chemin, réglages tels que stockés, et descripteur de sécurité décodé — propriétaire, et pour chaque entrée de la liste de contrôle d'accès son type, son masque, le SID et son nom. Les partages administratifs (`C$`, `ADMIN$`) n'y figurent pas : Windows ne les stocke pas.

`network_configuration.json`
: configuration réseau lue dans les ruches : nom d'hôte et domaine (`Global`), et pour chaque carte sa description et son pilote, une adresse matérielle forcée (`NetworkAddress`), son adresse matérielle courante et d'origine, sa dernière configuration IPv4 et IPv6, et son bail DHCP (obtention et fin, en UTC). Elle reste lisible quand la machine est débranchée ou le bail expiré, ce que le relevé vivant ne montre plus.

`event_log_configuration.json`
: configuration de chaque journal d'événements lue dans les ruches : les canaux (`WINEVT\Channels` : `Enabled`, `Type`, `OwningPublisher`, droits d'accès `ChannelAccess`, taille et rétention si elles sont fixées) et les journaux classiques (`Services\EventLog` : fichier, `MaxSize`, `Retention`, accès) et les sessions ETW lancées au démarrage (`WMI\Autologger` : `Start`, fichier, taille). Un intrus qui coupe un canal, réduit un journal, en ouvre l'accès ou arrête une session comme `EventLog-Security` y laisse sa trace ; `KeyLastWrite` date la dernière modification de chacun.

`fat_volumes.json`
: volumes FAT12, FAT16, FAT32 et exFAT, clés USB et cartes du suspect comprises (le volume de la collecte excepté) : chaque entrée, présente ou **supprimée** (`Deleted`), son chemin, son nom 8.3, sa taille, ses attributs et ses dates. Un nom supprimé en FAT garde son nom long s'il n'a pas été écrasé ; sinon son nom 8.3, première lettre perdue (`NameIncomplete`). Chaque date est donnée en UTC et en heure locale dérivée de l'UTC, comme dans tous les artefacts : un même instant se lit partout à la même heure. FAT stocke l'heure locale calculée par Windows au moment où il l'écrit : une date de modification antérieure à la création est une date conservée par une copie, écrite au moment de la copie, et convertie avec le décalage de ce moment (`ModifiedWrittenAt` `Copy`,

`ModifiedOffsetApplied`, `ModifiedAsStored`) — sans quoi un fichier d'hiver copié en été serait décalé d'une heure. exFAT garde son décalage (`ModifiedOffset`…), la date UTC est exacte. La collecte ne copie que les métadonnées du volume (secteur de démarrage, FAT, répertoires), pas le contenu des fichiers ; avec `--binary`, un programme de ces volumes cité par un artefact (lancé depuis une clé, par exemple) est empreinté et prélevé comme sur NTFS.

`srum.json`
: usage des ressources (SRUM, `sru\SRUDB.dat`), une entrée par ligne de chaque table : octets envoyés et reçus par application et par interface (une exfiltration s'y lit, avec le programme), cycles processeur, octets lus et écrits, connexions ; 30 à 60 jours. `Table` (GUID du fournisseur), `Provider` (nom documenté : `NetworkUsage`, `ApplicationResourceUsage`…), puis les colonnes par leur nom (lettres et chiffres seuls), et `App`, `UserSID`, `UserName` résolus par `SruDbIdMapTable`. La base est copiée vivante : la dernière heure est dans ses journaux `SRU*.log`, copiés dans la consigne pour un rejeu (`esentutl /r`) sur une copie.

`bits_jobs.json`
: travaux BITS (`qmgr.db`) : nom, description, type, état, propriétaire, **commande de notification** (lancée à la fin ou à l'échec du transfert : une persistance), délais, dates, et chaque fichier : URL source, destination, fichier temporaire, tailles. Le blob complet est donné en hexadécimal : BITS 10 y garde plus (le programme créateur) à des endroits non décrits.

`wmi_persistence.json`
: abonnements aux événements WMI, lus **hors ligne** dans le dépôt CIM copié en brut (`wbem\Repository`) : aucune requête WMI, rien exécuté sur la machine. Dans *chaque* espace de noms, chaque filtre (requête WQL), consommateur (ligne de commande, script VBScript ou JScript, journal, courriel, et toute classe qu'un intrus aurait définie), liaison et minuterie, avec toutes ses propriétés (`Properties` : nom et valeur ; `CreatorSID` en clair). C'est une persistance invisible au registre. Les deux dates d'en-tête d'une instance ne sont pas émises : ce ne sont pas des dates de création.

`web_client.json`
: navigation de chaque utilisateur dans les navigateurs Chromium (Edge, Chrome, Brave, Vivaldi, Chromium, Opera), lue dans la base `History` de chaque profil de navigateur. Un enregistrement par URL (`Type Url` : titre, nombre de visites et de saisies, dernière visite), par visite (`Visit` : instant, `Transition` — `typed` saisie, `link`, `reload`… — et ses qualificatifs, visite d'origine `FromUrl`, durée), par téléchargement (`Download` : chaîne complète des redirections `UrlChain`, page et onglet d'origine, chemin de destination, octets reçus, état, ouverture, `DangerType` brut, SHA-256 quand le navigateur l'a calculé) et par terme cherché dans la barre d'adresse (`SearchTerm`). Chaque enregistrement porte `Browser`, `BrowserProfile`, `SID` et `Source`. Les dates de Chromium (microsecondes depuis 1601, UTC) sont converties exactement. Les bases sont lues sur une copie privée avec leur journal, rejoué comme le ferait le navigateur. À côté de `History`, la consigne reçoit brutes les autres bases du profil (`Login Data`, `Web Data`, cookies, raccourcis, signets, préférences) et `Local State`, dont la clé protégée par DPAPI chiffre les mots de passe enregistrés ; jamais les caches. Firefox (`Browser Firefox`, chaque dossier de `Profiles`) donne les mêmes enregistrements depuis `places.sqlite` — visites avec leur type (`typed`, `link`, `download`…), téléchargements avec leur destination, leur fin et leur taille —, plus `TypedInput` (ce qui

a été tapé dans la barre d'adresse et la page atteinte) et, depuis `formhistory.sqlite`, `FormEntry` : chaque valeur saisie dans un formulaire ou la barre de recherche (`FieldName` `searchbar-history`), avec ses dates. Sont copiés bruts à côté : cookies, `logins.json` et `key4.db` (identifiants enregistrés, chiffrés), extensions, préférences, session. Enfin `TypedUrl` : chaque adresse tapée dans Internet Explorer ou la barre d'adresse de l'Explorateur (`TypedURLs` du `NTUSER.DAT`, `MruPosition` 0 pour la plus récente) avec l'instant de sa saisie (`TypedURLsTime`, `TypedUtc`). `SavedLogin` : chaque identifiant enregistré (`Login Data` de Chromium, `logins.json` de Firefox) — site, compte (en clair chez Chromium, chiffré et donc absent chez Firefox), création, dernier usage, nombre d'usages, refus d'enregistrer (`NeverSave`) ; `Cookie` : site, nom, chemin, dates, drapeaux. Les mots de passe et les valeurs des cookies, chiffrés, ne sont jamais décodés : les bases sont dans la consigne pour une analyse qui détiendrait la clé DPAPI de l'utilisateur. Et `CacheEntry` (`Browser` `WebCache`) : chaque entrée de la base `WebCacheV01.dat` d'Internet Explorer, d'Edge Legacy et des applications WinINet, par conteneur (`Container` : `History`, `MSHist…`, `Content`, `iedownload`, `BackgroundTransferApi`) — URL, compte et page d'une entrée d'historique (`User`, `Page`), nombre d'accès, dates, fichier du cache (`CachedFile` : son chemin, par le sous-dossier que l'entrée désigne). Le moteur d'Internet Explorer (WinINet) reste utilisé par `mshta.exe` et des scripts même quand IE est redirigé vers Edge : le cache de chaque profil (`INetCache\IE`) est collecté, limité aux types utiles à l'enquête — HTA, scripts, exécutables, archives, documents à macros, PDF —, et l'HTA qu'une attaque a fait exécuter s'y retrouve avec son entrée. Un téléchargement d'Internet Explorer 10 et 11 (conteneur `iedownload`) donne en outre `Download` : son URL, son type MIME, sa copie temporaire dans le cache, le chemin où il a été enregistré (`TargetPath`, ou seulement `SuggestedName` s'il ne l'a pas été), sa taille et l'adresse du serveur — format non documenté par Microsoft, établi sur un vrai téléchargement d'IE 11 dans la machine de test. Sur Windows 7 et antérieurs, Internet Explorer 5 à 9 tenait ces mêmes données dans des `index.dat` (cache, historique et ses dossiers `MSHist01`, cookies, téléchargements) : ils sont collectés et convertis en `CacheEntry` (`Browser` `Internet Explorer`), entrées récupérées (`Recovered`) et abandonnées (`RecordKind` `LEAK`) comprises ; l'heure de visite d'un historique hebdomadaire, locale, est convertie, l'expiration et la dernière vérification, dates FAT sans fuseau, sont données telles qu'écrites. La date de modification des conteneurs `MSHist`, que Windows écrit en heure locale, est convertie en UTC avec le décalage en vigueur à sa date. La base est copiée vivante avec ses journaux : ses dernières entrées peuvent n'être que dans ceux-ci (rejeu `esentutl /r` par l'analyste).

`dpapi_master_keys.json`

clés maîtresses DPAPI du système (`System32\Microsoft\Protect`, `Scope` `System`) et de chaque profil (`AppData\Roaming\Microsoft\Protect`, `User`). Les fichiers sont copiés tels quels dans la consigne, toujours chiffrés par le mot de passe de l'utilisateur ou par le secret DPAPI du système, avec `Preferred` et `CREDHIST` : avec ce mot de passe, l'analyse peut déchiffrer plus tard les secrets enregistrés (mots de passe des navigateurs, Wi-Fi, gestionnaire d'identifiants). Le JSON n'en donne que les métadonnées, jamais la clé : propriétaire (`SID`), `Guid` (celui de l'en-tête, vérifié égal au nom du fichier), version, politique, taille de chaque bloc — `DomainKeyLength` non nul : compte d'un domaine —, dates du fichier, et `Preferred` avec `PreferredExpiresUtc` pour la clé en usage.

`startup_folders.json`

contenu des dossiers de démarrage, de tous les utilisateurs (`Scope` `AllUsers`) et de chaque

profil (`User`, avec `SID`) : chaque fichier, ses dates et ses empreintes ; un raccourci est décodé (`Shortcut`, comme dans `recentdocs.json`).

`shortcuts.json`
raccourcis posés pour être cliqués, hors du dossier Recent : menu Démarrer de tous les utilisateurs et de chaque profil, Bureau de chaque profil et Bureau public, Quick Launch et éléments épinglés à la barre des tâches (`Location` : `StartMenu`, `Desktop`, `QuickLaunch`, `TaskBar` ; `Scope` et `SID` comme ci-dessus). Chaque raccourci est décodé comme dans `recentdocs.json` : cible, arguments, volume et machine d'origine. Un programme désinstallé y laisse son raccourci ; un intrus y pose une ligne de commande, ou redirige un raccourci existant vers son binaire en gardant l'icône. Le dossier Startup du menu Démarrer n'y figure pas : il relève de `startup_folders.json`. Un Bureau redirigé ailleurs sur un disque local (OneDrive, un autre volume, voir `user_shell_folders.json`) est lu là aussi.

`user_shell_folders.json`
dossiers de chaque utilisateur (`UserShellFolders` de son `NTUSER.DAT`) : nom (`Name`, et `FolderName` pour ceux qu'un GUID désigne, comme les Téléchargements), valeur telle qu'écrite (`Value`), chemin résolu (`Path`, absent pour un partage réseau) et date de dernière écriture de la clé. OneDrive déplace le Bureau et les Documents, une entreprise les redirige vers un partage : c'est là qu'il faut chercher les fichiers de l'utilisateur, et une redirection posée en silence est elle-même une trace.

`powershell_history.json`
historique PSReadLine de chaque profil et de chaque hôte (`Host` : `ConsoleHost`, l'ISE, VS Code) : les commandes dans l'ordre où elles ont été tapées, une commande sur plusieurs lignes réunie. Aucune date par commande : `SourceModified` date la dernière.

`activities_cache.json`
Chronologie Windows (`ActivitiesCache.db` de chaque compte de chaque profil, avec son journal `-wal`) : chaque ligne des tables `Activity` et `ActivityOperation` (`Table`), toutes ses colonnes. Sous Windows 10, les applications ouvertes, les fichiers et pages vus (`Payload` : titre, URI), le temps passé au premier plan (`Payload` `UserEngaged`, de `StartTime` à `EndTime`) et, si l'historique du presse-papiers est actif, ce qui a été copié (`ClipboardPayload`) ; sous Windows 11, qui a retiré la Chronologie, les seuls réglages synchronisés. Les dates sont des temps Unix en UTC, donnés en UTC et en heure locale ; les GUID dans l'ordre d'octets de .NET ; une date ou un GUID nul n'est pas émis. Les codes (`ActivityType`, `ActivityStatus`, `OperationType`) sont donnés tels quels : Microsoft ne les documente pas.

`search_index.json`
index de recherche Windows (Windows 11 : `Windows.db` et `Windows-gather.db`, avec leurs journaux `-wal`). Une fiche par élément indexé du magasin de propriétés (`Table` `PropertyStore`, `WorkId`) : chemin (`ItemUrl`, et `ItemPathDisplay` dans la langue de l'interface), dates de création, de modification, d'accès et d'indexation, taille, propriétaire, type et, pour un document, le début de son texte (`SearchAutoSummary`) ; chaque propriété sous son nom Windows sans le préfixe `System.`. Une fiche par document vu par le collecteur (`Table` `Gather`, `DocumentId`, le même numéro que `WorkId`) : son URL et sa dernière modification au passage du collecteur. L'enregistrement survit au fichier jusqu'au passage suivant : un document supprimé peut encore y paraître. Sous Windows 10, l'index (`Windows.edb`, ESE)

est collecté avec ses journaux mais pas converti.

`rdp_client.json`
connexions Bureau à distance *sortantes* de chaque profil : connexions récentes (`RecentConnection`, `MruPosition` 0 pour la plus récente) et serveurs contactés (`Server` : nom d'utilisateur proposé `UsernameHint`, empreinte du certificat accepté `CertHash`, dernière écriture de la clé), fichiers `.rdp` des Documents (`ConnectionFile`, `Default.rdp` compris, chaque réglage), et cache bitmap (`BitmapCache`) : chaque tuile de `Cache*.bin`, morceau de l'écran distant tel que le client l'a dessiné, est exportée en image dans `rdp_cache\<SID>\<fichier>`, avec sa taille, ses clés et le SHA-256 de ses pixels. L'ancien format `bcache*.bmc` est collecté mais pas décodé, faute d'échantillon.

`rdp_server.json`
connexions Bureau à distance *entrantes*, lues dans les journaux collectés : connexion reçue et authentification (RemoteConnectionManager 261 et 1149), connexion TCP acceptée (RdpCoreTS 131), sessions (LocalSessionManager 21 à 25, 39 à 41), ouvertures et fermetures de session de type 10 et reconnexions (Security 4624, 4625, 4634, 4778, 4779). Chaque événement garde ses données et reçoit `User`, `Domain`, `SourceAddress` et `SessionId`, quel que soit le nom que lui donne son journal. Peu utile sur un poste, c'est la première question sur un serveur.

`emails.json`
courriels d'Outlook de chaque profil : les PST (`Documents\Outlook Files`) et les OST, copie hors connexion d'un compte Exchange ou Microsoft 365 (`AppData\Local\Microsoft\Outlook`), lus par le code de WAC — PST Unicode d'Outlook 2003 et suivants, OST d'Outlook 2013 et suivants (pages de 4 Kio, blocs compressés). Une fiche par boîte (type, version, nombre de messages lus, de messages illisibles) et une par message : dossier, classe, objet, expéditeur, destinataires (`Recipients` : nom, adresse, To/Cc/Bcc), dates d'envoi, de remise, de création et de modification, identifiant Internet du message, en-têtes de transport. Le corps (texte, HTML) et chaque pièce jointe sont écrits à côté du JSON (`emails\<SID>\<fichier>\<message>\`), avec leur taille et leur SHA-256 ; le nom d'une pièce jointe, choisi par l'expéditeur, n'est jamais pris pour un chemin : le fichier porte son rang et, si elle est simple, son extension. Un message joint (un rendez-vous, un courriel transféré en pièce jointe) donne son objet et sa classe. Les fichiers qu'Outlook a ouverts depuis un message (`INetCache\Content.Outlook`) et les données locales du nouvel Outlook (`AppData\Local\Microsoft\Olk`) sont prélevés tels quels. Non décodés : le chiffrement « élevé » (cyclique) et le format ANSI d'Outlook 97 à 2002, sans échantillon pour les vérifier — la fiche de la boîte le dit. Thunderbird : ses profils (`AppData\Roaming\Thunderbird\Profiles`) sont prélevés entiers — carnet d'adresses et réglages des comptes compris — et chaque dossier de courrier, un fichier mbox, est décodé : en-têtes (mots encodés compris), date d'envoi en UTC, corps texte et HTML, pièces jointes (base64, quoted-printable, noms RFC 2231), dans les mêmes champs que pour Outlook, avec `Offset` (position du message dans le fichier) au lieu de `NodeId`.

`thumbcache.json`
caches des miniatures et des icônes de l'Explorateur, de chaque profil (`thumbcache_*.db`, `iconcache_*.db`) : ce que l'utilisateur a eu sous les yeux, même si le fichier a été supprimé depuis. Pour chaque entrée : son cache et sa taille, son hachage (`Hash` — le `System.ThumbnailCacheId` que Windows donne pour le fichier, ce qui relie

une miniature à son fichier), son identifiant, ses dimensions, son format (`png`, `jpg`, `bmp`), le SHA-256 de l'image, et l'image elle-même, exportée à côté du JSON (`thumbcache\<SID>\<cache>\<hachage>.<ext>`). Les deux sommes de contrôle que Windows écrit pour chaque entrée sont recalculées (`HeaderChecksumValid`, `DataChecksumValid`) : une entrée lue de travers les échoue. Windows 8 à 11 ; la disposition de Vista et 7 n'est pas décodée.

`error_reports.json`
rapports d'erreurs Windows (`Report.wer`) de la machine et de chaque profil : programme, chemin, instant (`EventTimeUtc`), signature du problème (module fautif, version), toutes les valeurs du rapport dans `Values`.

`defender_log.json`
journaux de support de Microsoft Defender (`MPLog-*.log`) : chaque ligne telle qu'écrite, datée quand elle commence par une date UTC. Analyses à l'exécution, détections, exclusions, mises à jour. Les traces WPP du même dossier (`MpWpp*.bin`, traces ETW binaires que seuls les symboles privés de Microsoft décodent) sont copiées brutes dans la consigne, sans conversion.

`defender_detections.json`
une entrée par fichier DetectionHistory, qui survit à l'effacement du journal d'événements : menace, ressource (fichier, clé), compte, processus, date de détection initiale, entrée de quarantaine, et les propriétés nommées `ThreatTracking` (empreintes, taille, heure de début). Le format n'est pas documenté : seuls les champs dont la position a été confirmée contre `MSFT_MpThreatDetection` sont nommés ; les autres sont rendus tels que lus (`UnnamedFields` : position, type, valeur), sans signification inventée.

`defender_quarantine.json`
entrées de la quarantaine : menace, instant de mise en quarantaine, chemin d'origine du fichier retiré, ses dates de création et de modification, ses attributs, sa taille, et l'empreinte de son contenu (`ContentSha256`). La quarantaine est copiée **chiffrée**, telle que Defender la garde : rien de malveillant n'est écrit en clair, ni sur le support de collecte ni sur le poste d'analyse. Le contenu n'est déchiffré qu'en mémoire, pour son empreinte.

`hosts.json`
entrées du fichier `hosts` : adresse et noms, numéro de ligne ; un Windows intact n'en a aucune.

6.5.3 Fichiers et navigation

`mft.json`
chaque fichier et répertoire de chaque volume NTFS fixe (`Volume` : `C:`, `G:`…; un numéro d'enregistrement n'a de sens que dans son volume), présent ou **supprimé** (`Deleted`) tant que NTFS n'a pas réutilisé son enregistrement : chemin reconstitué, taille, attributs, et deux jeux de quatre dates en UTC — `CreatedUtc`… (attribut `$STANDARD_INFORMATION`, ce que montre l'Explorateur et que réécrit un outil d'antidatage) et `FileNameCreatedUtc`… (attribut `$FILE_NAME`, écrit par NTFS seul). Une date `$STANDARD_INFORMATION` antérieure à celle de `$FILE_NAME` trahit un antidatage. `PathIncomplete` marque un chemin dont un répertoire parent a été réutilisé depuis : le chemin n'est pas inventé. Un fichier à liens physiques

(`HardLinks`) est un seul enregistrement sous plusieurs noms — `System32\crypt32.dll` et son original sous `WinSxS` : `HardLinkPaths` donne ses autres chemins, chacun cherchable. `SecurityId` est le numéro de son descripteur de sécurité dans `security_descriptors.json`.

`security_descriptors.json`
descripteurs de sécurité de chaque volume NTFS fixe (`$Secure:$SDS`) : chacun une seule fois, quel que soit le nombre de fichiers qui le partagent, repéré par `Volume` et `SecurityId` — la clé que porte chaque fichier de `mft.json`. Il donne le **propriétaire** (`Owner`, `OwnerName`), le groupe, et chaque entrée de la DACL et de la SACL (type, drapeaux, masque d'accès, SID et nom). L'outil d'un intrus déposé par son compte, un fichier système qui n'appartient plus à TrustedInstaller, une ACL ouverte à Everyone s'y lisent, fichiers supprimés compris. NTFS range avec chaque descripteur un condensat de ses octets : WAC le recalcule, et `HashMismatch` signale un descripteur qui ne lui correspond pas.

`usn_journal.json`
journal des modifications de chaque volume NTFS fixe (`$UsnJrnl:$J`, `Volume` ; les numéros `Usn` croissent au sein d'un volume) : chaque création, écriture, renommage et **suppression**, daté, avec ses motifs (`Reasons`, un bit inconnu rendu en hexadécimal) et le chemin reconstitué par la `$MFT`. Il garde la trace de fichiers déposés puis effacés, que la `$MFT` a déjà oubliés. La `$MFT`, le journal des modifications, `$LogFile` (journal de transactions, non converti) et `$TxfLog` (journal de NTFS transactionnel, journal CLFS, ses conteneurs et le flux `$Tops:$T`, non converti) sont en outre copiés bruts dans la consigne, pour refaire plus tard la chronologie avec un autre outil.

`ntfs_volumes.json`
chaque volume NTFS fixe, lu dans son secteur d'amorçage (`$Boot`, reconnu à ses signatures avant tout champ) : numéro de série (`VolumeSerial`, les 32 bits que les raccourcis, listes de sauts et fichiers Prefetch relèvent du volume de leur cible, et `VolumeSerial64` en entier), tailles de secteur, de cluster, d'enregistrement et de bloc d'index, emplacement de la `$MFT` et de son miroir. Puis l'en-tête du journal des modifications (`$UsnJrnl:$Max`, `UsnJournal`) : taille maximale, pas d'allocation, `UsnJournalId`, qui est la date de **création** du journal (`Created`), et premier numéro encore présent. Un journal supprimé puis recréé (`fsutil usn deletejournal`) se lit à une création bien postérieure à l'installation.

`recentdocs.json`
raccourcis de documents récents : cible, horodatages du raccourci et de la cible, taille de la cible, volume (type, numéro de série, nom), chemin réseau, liste d'éléments du shell, et les blocs de données supplémentaires : le bloc de suivi (`Tracker` : nom NetBIOS de la machine où était la cible, identifiants de suivi, et ce qu'en révèle le GUID de version 1 — la date de début du suivi et l'adresse matérielle de la machine), chemins avec variables d'environnement, dossier connu, propriétés de la cible.

`jumplistAutomaticDestinations.json`, `jumplistCustomDestinations.json`
listes de sauts par application, avec les raccourcis qu'elles contiennent ; pour les listes automatiques, chaque raccourci est accompagné de son entrée DestList (dernier accès, nom de la machine, identifiants de suivi avec leur date et leur adresse matérielle, épinglage, nombre d'ouvertures `InteractionCount`, rang `MruPosition` — 0 pour le plus récent —, nom du dossier connu visé).

`shellbags.json`
dossiers parcourus dans l'Explorateur, arborescence reconstituée ; pour chaque dossier, son rang parmi ses voisins (`MruPosition`, 0 pour le plus récent), la date d'écriture de la clé qui le porte (`LastWriteTime`) et de sa propre clé (`KeyLastWriteTime` : son ajout, ou le parcours d'un sous-dossier), et son `NodeSlot`. Les « première » et « dernière interactions » d'autres outils ne sont que des lectures de ces deux dates.

`mrus.json`, `mruApps.json`
listes de fichiers et d'applications récemment utilisés.

`zone_identifiers.json`
flux `Zone.Identifier` (« marque du web ») que navigateurs et messageries écrivent sur ce qu'ils téléchargent : fichier, zone (`ZoneId`, `ZoneName` — `Internet` pour 3), page d'origine (`ReferrerUrl`) et adresse du fichier (`HostUrl`), toutes les valeurs dans `Values`. Lus dans la copie de la `$MFT`, où ce petit flux est résident : aucune lecture de plus sur la machine examinée ; un flux non résident, rare, est signalé (`ContentNotResident`).

`recycle_bin.json`
fichiers mis à la corbeille, sur chaque volume NTFS fixe, d'après leur index `$I` : chemin d'origine, taille, instant de la suppression (`DeletedUtc`), et l'utilisateur (`SID`). Seuls les `$I` sont prélevés ; les fichiers eux-mêmes (`$R`) figurent dans `mft.json`.

6.5.4 Périphériques

`setupapi.json`
sections de `setupapi.dev.log` et de ses rotations : opération (`Operation`), périphérique ou pilote (`Subject`, avec le numéro de série d'une clé USB), début et fin convertis depuis l'heure locale de la machine examinée, résultat. La première connexion d'un périphérique s'y date, même quand la clé `USBSTOR` a été nettoyée.

`Usbstor.json`
périphériques de stockage USB connus du système : `Vendor`, `Product`, `Revision` et `Type`, tirés de `DeviceId` ; `SerialNumber`, tiré de `InstanceId` — absent quand l'identifiant a été fabriqué par Windows (deuxième caractère « & »), le périphérique n'en déclarant aucun ; dates de première et de dernière insertion.

`mounted_device.json`
correspondance entre lettres de lecteur et périphériques. `Drive` nomme la lettre ou le volume ; `Type` dit la forme de la valeur : `GPT partition` (`PartitionGuid`), `MBR partition` (`DiskSignature` et `PartitionOffset`, en octets), `Device path` (`Device`, chemin du périphérique, avec le numéro de série d'une clé USB) ou `Unknown` (`Data`, les octets en hexadécimal).

6.5.5 Journaux d'événements (`--events`)

`events.json` contient un objet par événement :

`EvtSystem*`
: la section système : fournisseur, identifiant, niveau, tâche, mots-clés, horodatage de création, numéro d'enregistrement, processus et fil, canal, machine, utilisateur ;

`EvtEventData`
: les données de l'événement, en liste de paires `Name` / `Value` ;

`EvtEventMessage`
: le message lisible, tel que l'Observateur d'événements l'afficherait, reconstitué à partir des ressources du fournisseur prélevées sur la machine : manifeste (`WEVT_TEMPLATE`, ou son jumeau `SystemResources\*.mun` sous Windows 11) et table de messages dans la langue de l'interface de la machine, ou fichiers `EventMessageFile` d'une source classique. Chaque donnée y est montrée comme Windows la montre : libellé de sa table de valeurs ou de ses drapeaux, hexadécimal, texte d'un code d'erreur Win32, HRESULT ou NTSTATUS, adresse réseau ;

`EvtSourceLog`
: le fichier journal dont provient l'événement. Un même canal peut être porté par plusieurs fichiers (journal courant et archives) dont les numéros d'enregistrement se recouvrent ;

`EvtRecovered`
: `true` pour un événement d'un journal *vidé* : Windows remet l'en-tête du bloc à zéro sans effacer les enregistrements, que WAC relit. L'Observateur d'événements ne les montre plus ; ils restent la trace de ce que l'effacement voulait faire disparaître, et ne doivent jamais être lus comme des événements vivants.

> **Messages incomplets**
>
> Une marque `%1`, `%2`… restée dans un message signifie que l'événement ne porte pas la donnée attendue : elle est laissée visible pour que la phrase ne paraisse pas complète. Tous les fournisseurs ne livrent pas de ressources de messages : leurs événements n'ont alors pas de `EvtEventMessage`, mais leurs données restent complètes.

CHAPITRE 7

Lire les données

7.1 Horodatages

Chaque date est émise deux fois : `X` en heure locale et `XUtc` en UTC. L'heure locale est celle du **suspect**, calculée avec le fuseau lu dans sa ruche `SYSTEM` — pas celle de la machine de collecte. Les dates portent la fraction de seconde quand la source la fournit, et seulement dans la précision de cette source : sept chiffres pour un `FILETIME`, trois pour une date à la milliseconde, aucun pour une date à la seconde (date FAT, précise à deux secondes, date lue en texte, temps Unix).

Chaque date porte le décalage en vigueur **à cette date**, heure d'été comprise : une date de janvier relevée en été à Paris est à `+01:00`, une date de juillet à `+02:00`. `WAC` applique les règles de la machine examinée année par année (clé `Dynamic DST` de la ruche `SOFTWARE`) : une date de 2005 aux États-Unis suit les règles de 2005. L'heure locale écrite est toujours dérivée de l'heure UTC : un même instant se lit à la même heure dans tous les artefacts. Deux dates n'ont aucun fuseau connaissable et sont données **telles qu'écrites**, sans décalage ni UTC, avec `TimeZone` qui le dit : la date d'un élément vu dans une archive ZIP (`ModifiedAsWritten` ; une entrée ZIP garde l'heure d'horloge de la machine qui a fait l'archive — les archives que Windows 11 lit lui-même, 7z ou tar, portent au contraire un instant UTC, donné comme tel) et une date OLE d'un magasin de propriétés (`AsWritten`). Les placer sur une chronologie demande de connaître leur origine : `WAC` ne l'invente pas. Pour une valeur que l'artefact enregistre en heure locale (volume FAT, `setupapi.dev.log`), l'heure répétée à l'automne est lue comme sa première occurrence, et l'heure sautée au printemps avec le décalage d'avant le changement, comme le fait Windows.

7.2 Noms des comptes

Les champs qui nomment un compte (`SIDName`, `Owner`, `RunAs`) sont tirés des preuves seules, sans interroger le système ni le réseau : comptes et groupes locaux de la base SAM (dans la langue de la machine examinée), SID bien connus sous leur nom canonique (`SYSTEM`, `LOCAL SERVICE`…), SID de services (`NT SERVICE\…`), et dossier de profil pour un compte de domaine. Un SID qu'aucune de ces sources ne nomme est publié sans nom : le SID reste la preuve, à rapprocher du domaine par l'analyste.

Le profil Default. `C:\Users\Default` est le modèle copié pour chaque nouveau profil : une valeur Run, un raccourci ou une association posés là atteignent tous les comptes créés ensuite. WAC le lit comme un profil (ruches `NTUSER.DAT` et `UsrClass.dat`, Bureau, menu Démarrer, Recent…). N'appartenant à aucun compte, il n'a pas de SID : ses entrées portent le SID `S-0-0-0`, fait de zéros, au format d'un SID pour que les outils les lient, mais qui n'en

est pas un (révision 0), et `Profile` : `Default` à la place de `SIDName`. Ce n'est pas `S-1-0-0`, le vrai SID nul de Windows.

7.3 Noms des GUID

Les champs qui nomment un GUID (classe COM, dossier connu, gestionnaire d'une tâche) viennent d'abord d'une table de référence, puis des ruches de la machine examinée : ses classes (`SOFTWARE\Classes\CLSID`), ses dossiers connus (`FolderDescriptions`), et les classes de chaque utilisateur (`UsrClass.dat`). Un composant installé sur la machine y est donc nommé, avec le module qui le sert. Un GUID que rien ne nomme est publié sans nom.

7.4 Un champ absent n'est pas un champ vide

Une valeur sans source n'est pas émise du tout. Une clé vide se lirait comme une lecture ratée, et ne se distinguerait pas de « la ruche ne contient pas cette valeur ». Ce qui n'a pas pu être lu est consigné dans `investigation.json`.

7.5 Artefact non collecté

Quand un artefact entier n'a pas pu être lu, son fichier le dit explicitement :

```
{
  "Artifact": "USBSTOR (registre)",
  "CollectionStatus": "NotCollected",
  "Error": "0x02 Le fichier spécifié est introuvable.",
  "Note": "La lecture de cet artefact a échoué : l'absence de données
           ci-dessus ne signifie PAS qu'aucune trace n'existe sur le système."
}
```

> **Attention**
> Un fichier d'artefact vide et un fichier « NotCollected » ne disent pas la même chose. Le premier indique qu'aucune entrée n'a été trouvée ; le second, que la source n'a pas pu être lue.

7.6 Données non décodées

Un élément que WAC ne sait pas décoder (élément du shell inconnu, bloc d'extension non reconnu) est accompagné de ses octets bruts en hexadécimal, pour qu'un analyste puisse le décoder plus tard. Avec `--dump`, la valeur brute des shellbags et des raccourcis est ajoutée systématiquement.

7.7 Chemins

Les chemins et valeurs sont restitués **tels qu'ils figurent sur le disque**, sans transformation : l'échappement n'intervient qu'à l'écriture du JSON.

CHAPITRE 8

Traces laissées sur la machine examinée

À lire avant d'employer WAC sur une affaire. Ce chapitre énumère ce que WAC fait à la machine, opération par opération, y compris ce qu'il ne peut pas éviter.

8.1 Ce que WAC fait encore

Opération	Trace
Lecture directe du volume	Un accès par volume effectivement lu (en général un seul). Aucun fichier ouvert, donc aucune date d'accès modifiée. Si l'audit d'accès aux objets est activé, l'ouverture du volume peut être journalisée (Security 4656/4663).
Écriture des résultats	Sur le support de collecte uniquement. Rien n'est écrit sur le disque examiné.
Rejeu des journaux de ruche	Sur la copie de travail uniquement, avec journal d'annulation.
État des services	Une requête en lecture au gestionnaire de services. Aucun service n'est ouvert ni modifié.
Processus	Un instantané du noyau ; aucun processus n'est ouvert. Relevé des propriétaires : une requête au service Terminal Services.
Sessions	Une requête en lecture à LSASS.
État réseau	Lectures en mémoire des tables de la pile TCP/IP : aucun paquet émis, aucun nom résolu. `iphlpapi.dll` n'est chargée que si `artefacts.network` est activé.
Heure de démarrage	Une requête en mémoire au noyau.
Journaux d'événements (`--events`)	Lus par le même accès direct au volume : le service de journalisation n'est pas sollicité.
Fichiers cités (`--binary`)	Lus par le même accès direct. Aucun fichier ouvert.
Vérification d'authenticité (`--binary`)	Catalogues lus par le même accès direct ; vérification en mémoire. Aucun service sollicité, aucun accès au magasin de certificats, aucun accès réseau.

8.2 Ce que WAC ne fait plus

Les versions antérieures créaient un cliché instantané (VSS), passaient par COM et WMI, interrogeaient le planificateur de tâches et le service de journalisation, ouvraient chaque processus et chaque service, lisaient la liste des profils dans le registre vivant et ouvraient chaque fichier cité pour en calculer l'empreinte. Toutes ces opérations, qui laissaient des entrées dans les journaux ou modifiaient des dates d'accès, ont été supprimées.

8.3 Ce qu'aucun outil ne peut éviter

Ces traces sont listées pour que l'analyste reconnaisse l'empreinte de WAC au lieu de la prendre pour l'activité étudiée :

— **lancer `WAC.exe` est une exécution** : elle produit un fichier Prefetch pour `WAC.exe`, une entrée Amcache et, si l'audit de création de processus est actif, un événement Security 4688. Attendez-vous à retrouver WAC dans les artefacts qu'il collecte ;

— **brancher la clé USB** l'inscrit dans `USBSTOR`, `MountedDevices` et les journaux associés ;

— **ouvrir une session** pour lancer l'outil crée une session et ses entrées de journal ;

— WAC **occupe de la mémoire** pendant la collecte, ce qui peut évincer des pages vers le fichier d'échange. Sa lecture du volume, elle, contourne le cache (lecture non tamponnée) : les gigaoctets lus ne remplacent pas le cache de fichiers de la machine.

CHAPITRE 9

Limites connues

Volumes ni NTFS ni FAT
Un fichier situé sur un CD-ROM n'est pas lu : il est signalé comme non lu.

Variables propres à un utilisateur
Un chemin exprimé avec `%APPDATA%`, `%LOCALAPPDATA%` ou `%USERPROFILE%` (dans une tâche planifiée, par exemple) n'est pas résolu : le compte concerné n'est pas connu à cet endroit, et deviner donnerait l'empreinte d'un autre fichier.

Exécutable supprimé
Un processus dont l'exécutable a été effacé du disque n'a pas d'empreinte : il ne reste que la mémoire, hors du périmètre de WAC.

Messages d'événements
Sur la machine de test, 99,6 % des événements reçoivent un message lisible, identique à celui de Windows pour 98 % des sortes d'événements. Le reste est expliqué dans le dossier de validation : écarts de Windows lui-même par rapport au manifeste, satellites de langue plus anciens que leur binaire. Les données, elles, sont toujours complètes.

Signés mais non authentifiés
Quelques sortes de fichiers réellement signés restent prélevées, dans le sens prudent : les applications du Microsoft Store (signées au niveau du paquet, et non fichier par fichier) et les scripts Windows Script Host signés seulement dans leur texte.

CHAPITRE 10

Dépannage

Message ou constat	Cause et remède
Échec de « Check administrator rights »	L'invite n'a pas été lancée en tant qu'administrateur. Relancez-la avec les droits d'administrateur.
Échec de « Check OS >= Windows 10 »	Système antérieur à Windows 10 : non pris en charge.
Échec de « Checking the collection medium », répertoire non vide	Le dossier de sortie contient une collecte antérieure. Indiquez un dossier neuf avec `--output`.
Échec de « Checking the collection medium », place insuffisante	Le support est trop petit pour les options choisies (section 3.2). Utilisez un support plus grand ou retirez `--binary`.
Un fichier d'artefact porte `"CollectionStatus": "NotCollected"`	La source n'a pas pu être lue ; le motif est dans `Error`. Les autres artefacts ne sont pas affectés.
Des pièces en échec au manifeste	Fichiers cités absents du disque (logiciel désinstallé, profil de service sans ruche), volume non NTFS… Le motif est dans `Result`.
Le journal indique des binaires « hachés sans être copiés »	La place a manqué pendant le prélèvement : les empreintes sont complètes, les copies non.
Besoin de comprendre un échec de lecture	Relancez avec `--loglevel=2` (et, pour la lecture NTFS, `--debug`) et consultez `WAC.exe.log`.

ANNEXE A

Glossaire

Artefact
Trace laissée par l'activité du système ou d'un utilisateur, et exploitable en investigation.

Consigne
Répertoire des copies brutes, scellé, jamais réécrit.

Empreinte
Valeur calculée sur le contenu d'un fichier (MD5, SHA-1, SHA-256) : deux contenus différents ont des empreintes différentes.

Journal de transaction
Fichier `.LOG1`/`.LOG2` contenant les modifications récentes d'une ruche, pas encore écrites dans celle-ci.

Manifeste
Fichier qui identifie chaque pièce de la consigne.

`$MFT`
Table des fichiers d'un volume NTFS.

Ruche
Fichier constituant une partie de la base de registre (`SYSTEM`, `SOFTWARE`, `ntuser.dat`…).

Sceau
Empreinte du manifeste, conservée à part : elle révèle toute retouche du manifeste.

Travail
Répertoire des copies de travail, sur lesquelles portent l'analyse et les éventuelles modifications.

WOF / Compact OS
Mode de stockage compressé des fichiers système de Windows 10 et 11.